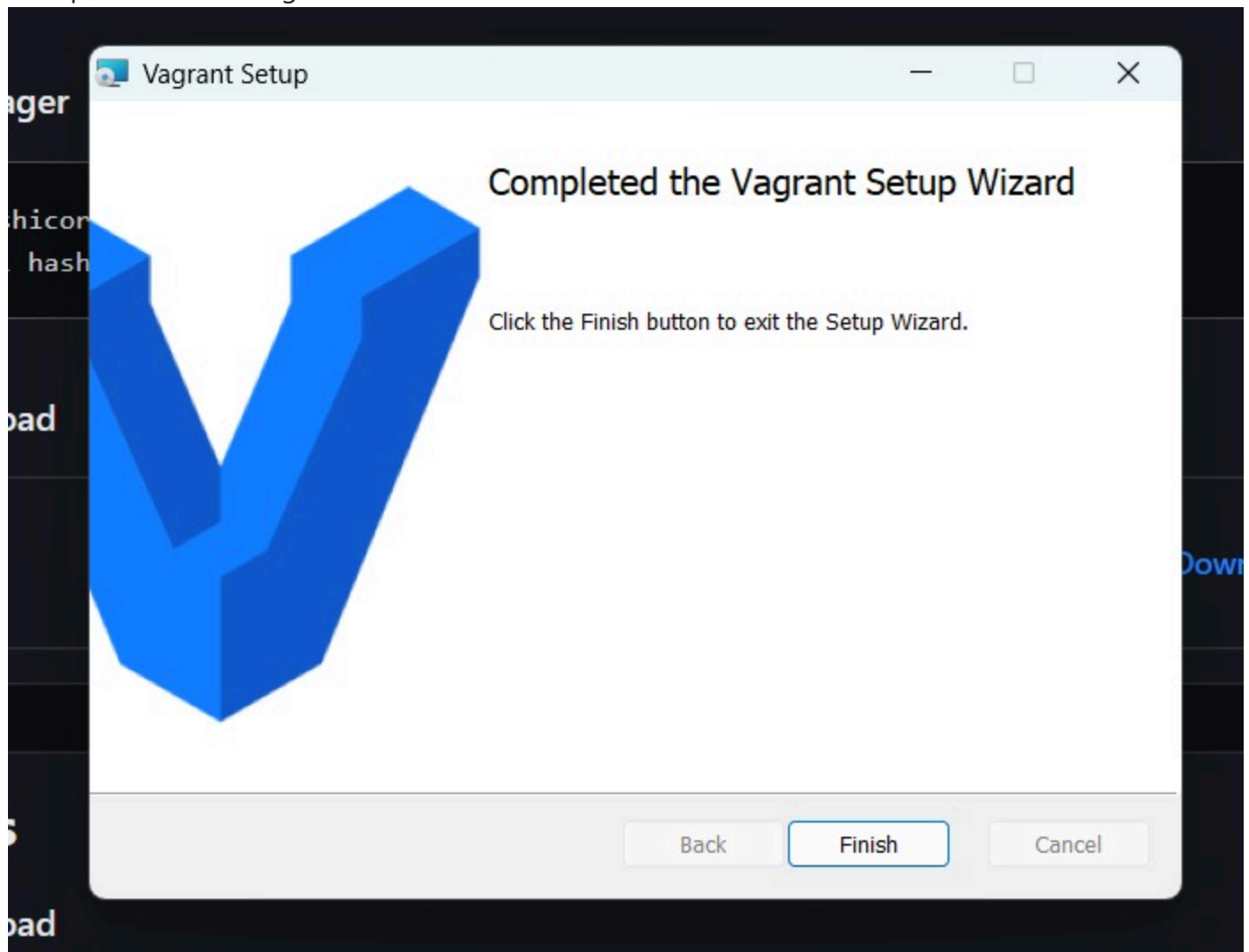


Containerization_and_DevOps

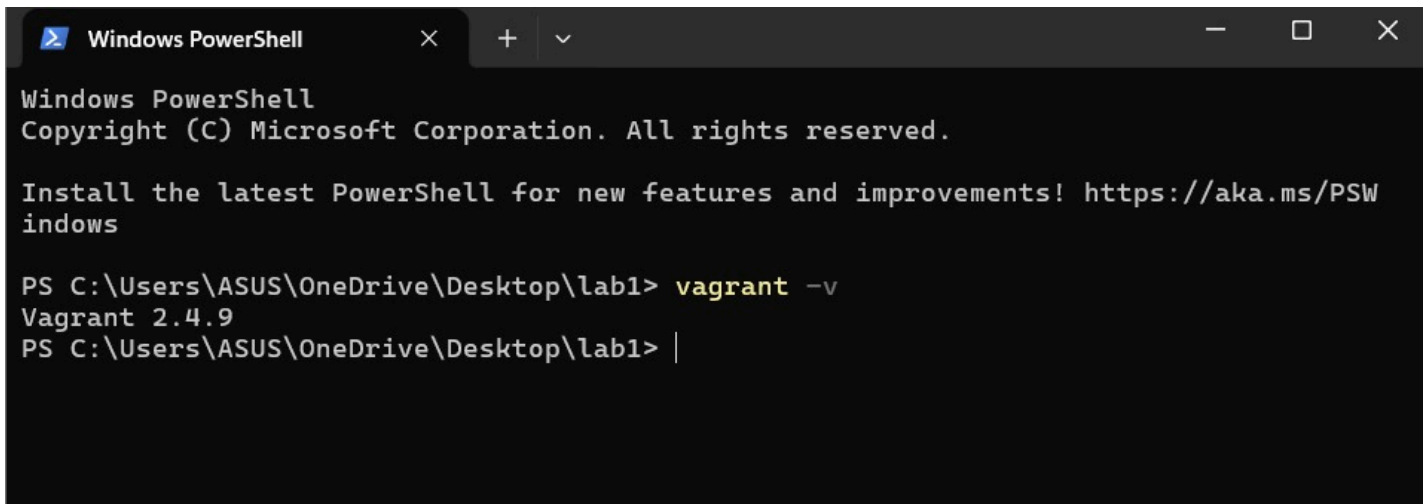
Step 1: Download Virtual Box from [here](#)

**Step 2:Download Vagrant from [here](#)



Step 3:To verify the installation we will check the version via following command

```
vagrant --version
```



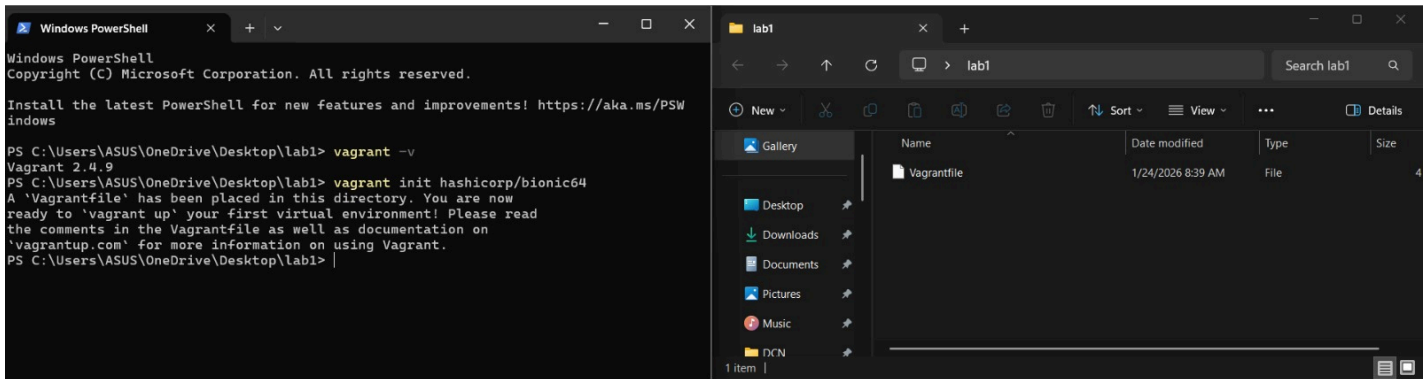
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSW
indows

PS C:\Users\ASUS\OneDrive\Desktop\lab1> vagrant -v
Vagrant 2.4.9
PS C:\Users\ASUS\OneDrive\Desktop\lab1> |
```

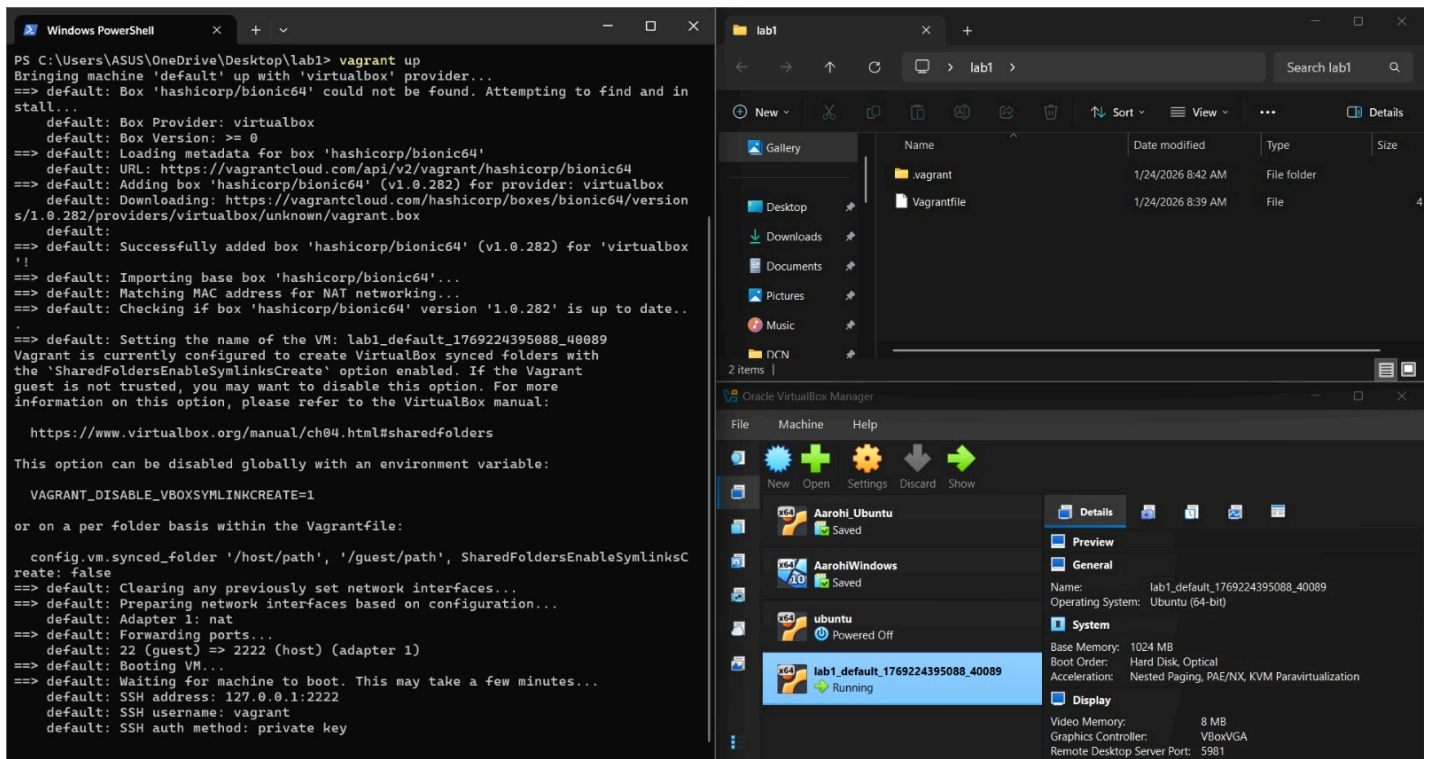
Step 4:Initialize Vagrant with Ubuntu box:

```
vagrant init hashicorp/bionic64
```



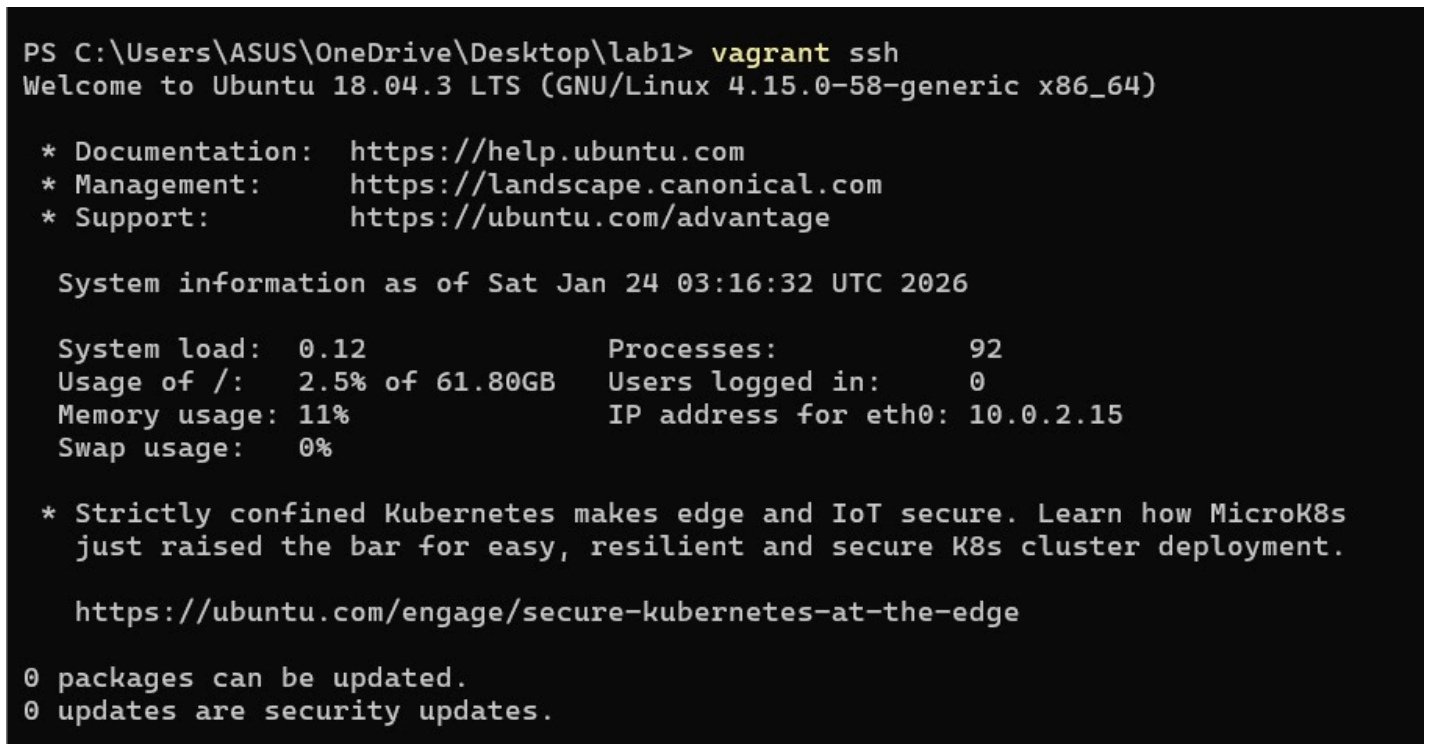
Step 5:Start the VM:

```
vagrant up
```



Step 6: Access the VM:

```
vagrant ssh
```



Step 7: Install Nginx inside VM

```
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
```

```
C:\Users\ASUS>wsl
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ sudo apt update
[sudo] password for aaroHi26:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1699 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1409 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [314 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.0 kB]
```

Step 8: Verify Nginx

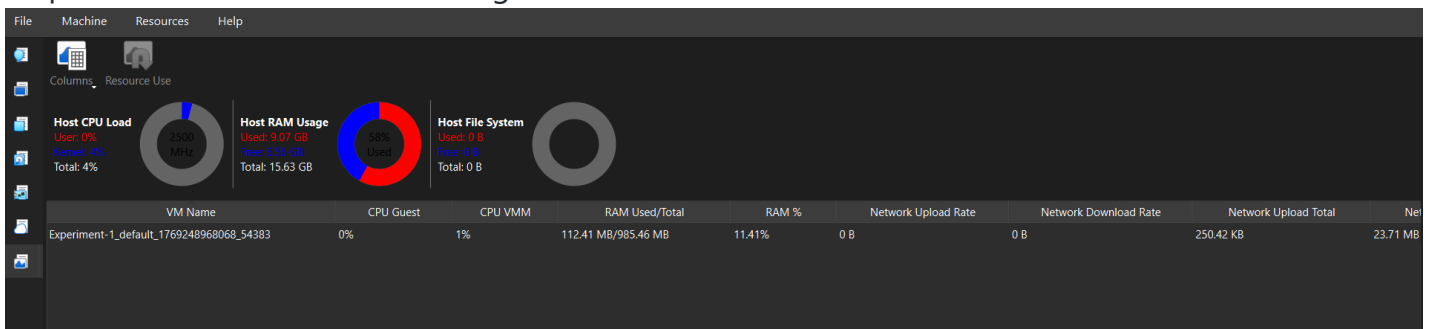
```
curl localhost
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

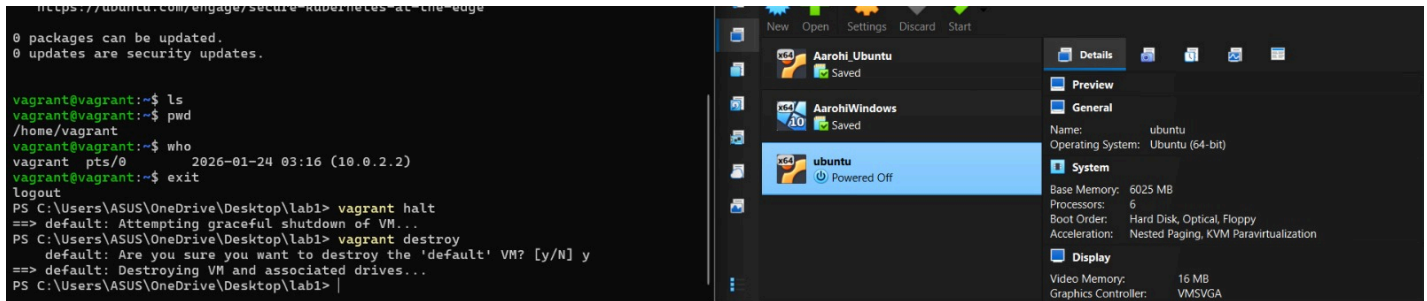
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Step 9: Utilization Matrix In Running State

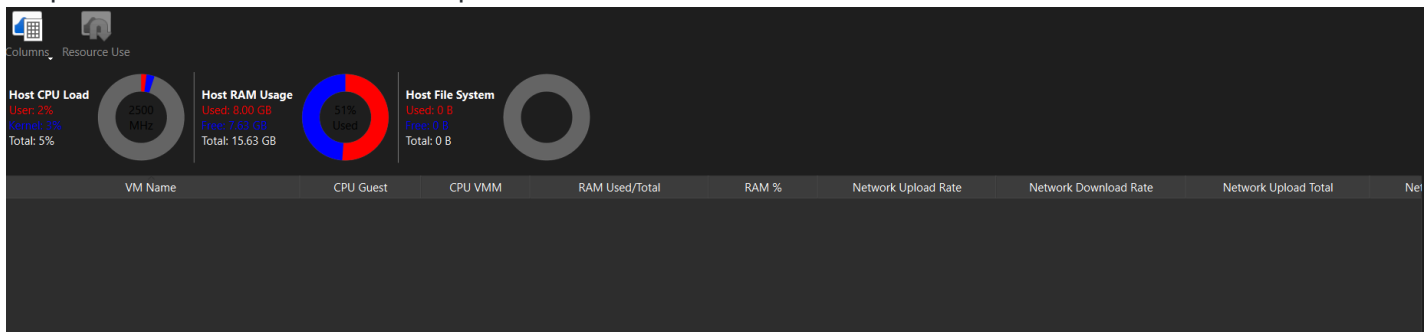


Step 10: Stop VM

```
vagrant halt
```

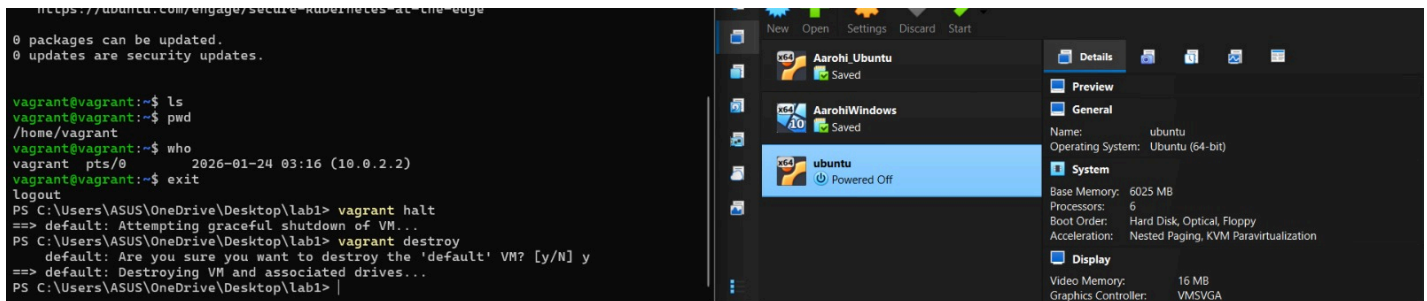


Step 11: Utilization Matrix In Stop State



Step 12: Remove VM

```
vagrant destroy
```



Containerization_and_DevOps

Experiment 2:- Docker Installation, Configuration, and Running Images

Step 1: Pull Image

```
docker --version
```

```
C:\Users\ASUS>wsl
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Step 2: Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d -p 8080:80 nginx
7af99e29809521934994293cf3c19e26a7dcf00d758917df9d722831fcfb3619
```

Step 3: Verify Running Containers

```
docker ps
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS                               NAMES
7af99e298095   nginx    "/docker-entrypoint..." 11 seconds ago Up 10 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp dazzling_rosalind
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Step 4: Stop Container

```
docker stop <container_id>
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker stop 7af99e298095
7af99e298095
```

Step 5: Remove Container

```
docker rm <container_id>
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker rm 7af99e298095
7af99e298095
```

Step 5: Remove Image

```
docker rmi nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker rmi nginx
Error response from daemon: conflict: unable to remove repository reference "nginx" (must force) - container c6b89ec4c72f is using its referenced image 4af177a024eb
```

Result Docker images were successfully pulled, containers executed, and lifecycle commands performed.

Containerization_and_DevOps

Experiment 3:- Docker Deploy NGINX Using Different Base Images and Comparing Image Layers

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

Step 1: Pull the Image

```
docker pull nginx:latest
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker pull nginx:latest
latest: Pulling from library/nginx
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d --name nginx-official -p 8080:80 nginx
2c2a5d3d1bfe39a727e3dd8a677d925c92bd1889cbe9ac6a2f55c04b9d097ebe
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Step 3: Verify

```
curl http://localhost:8080
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

You should see the NGINX welcome page.

Key Observations

docker images nginx

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker images nginx
REPOSITORY      TAG          IMAGE ID      CREATED        SIZE
nginx            latest       341bf0f3ce6c  2 weeks ago   240MB
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

- Image is pre-optimized
- Minimal configuration required
- Uses Debian-based OS internally

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

```
FROM ubuntu:22.04
```



```
RUN apt-get update && \  
    apt-get install -y nginx && \  
    apt-get clean && \  
    rm -rf /var/lib/apt/lists/*
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir nginx-ubuntu  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ cd nginx-ubuntu  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ pwd  
/mnt/c/Users/ASUS/nginx-ubuntu  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ nano Dockerfile  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ ls  
Dockerfile  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ |
```

```
GNU nano 7.2  
FROM ubuntu:22.04  
  
RUN apt-get update && \  
    apt-get install -y nginx && \  
    apt-get clean && \  
    rm -rf /var/lib/apt/lists/*  
  
EXPOSE 80  
  
CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ docker build -t nginx-ubuntu .
[+] Building 90.0s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 218B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a9751
=> => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a9751
=> => sha256:b1c2a2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed 29.54MB / 29.54MB
=> => extracting sha256:b1c2a2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lists/*
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:94d5856ec8a82c0230f62ec767cf2cf301fe48982f170926e29563b3fb3eaf02
=> => exporting config sha256:c97ef0abbcc3408b7d195812fd728121d2ef5929efa7924a0f63d0a20432bb2c
=> => exporting attestation manifest sha256:b43cb80a09b5dbe06553c7b425b216b4c84b89a14278851fc76c754ff2233da1
=> => exporting manifest list sha256:5fb9b602c9b178a136c69b470557fb86331a4fc48dff2c27126bd20344cbad52
=> => naming to docker.io/library/nginx-ubuntu:latest
=> => unpacking to docker.io/library/nginx-ubuntu:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ |

```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

```
``` /
```

![[Run Container](/Containerization\_and\_DevOps/Labwork/Experiment3/Images/Img8.png)]

**\*\*Observations\*\***

```
``` bash
```

```
docker images nginx-ubuntu
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ docker images nginx-ubuntu
REPOSITORY      TAG         IMAGE ID      CREATED        SIZE
nginx-ubuntu     latest      5fb9b602c9b1  About a minute ago  199MB
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ |

```

- Much larger image size
- More layers
- Full OS utilities available

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest
```

```
RUN apk add --no-cache nginx
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-ubuntu$ cd ..
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir nginx-alpine
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ cd nginx-alpine
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ nano Dockerfile
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ |
```

```
GNU nano 7.2 Dockerfile *
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ docker build -t nginx-alpine .
[+] Building 36.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 133B
=> [internal] load metadata for docker.io/library/alpine:latest
=> [auth] library/alpine:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
=> => sha256:589002ba0eae121aldbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB
=> => extracting sha256:589002ba0eae121aldbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153
=> [2/2] RUN apk add --no-cache nginx
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:f1f129f89b425a958ae2c45acd1cc0d89a8b07aea44109c1537219646e38a2d9
=> => exporting config sha256:celfad64dc73e10e3342003bdcc0c87cbaada70fcb57ac6dd51aa778b5786ba
=> => exporting attestation manifest sha256:8c85bd8a7402c41eed1d40ba823e00c4e0df3ef941845163ea8e267de5d523e5
=> => exporting manifest list sha256:f9f17069187b557e7cdcb0a58e36ef0257e8e06e3f07ae4eed126970b1da3995
=> => naming to docker.io/library/nginx-alpine:latest
=> => unpacking to docker.io/library/nginx-alpine:latest
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ |
```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
985e37fb8e096a87886d33a80e000427b85e4795dc54bc073b8cbcd50dcf6b3
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ |
```

Observations

```
docker images nginx-alpine
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ docker images nginx-alpine
REPOSITORY      TAG         IMAGE ID      CREATED        SIZE
nginx-alpine    latest     f9f17069187b  53 seconds ago 16MB
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ |
```

- Extremely **small image**
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ docker images | grep nginx
nginx-alpine    latest     f9f17069187b  About a minute ago 16MB
nginx-ubuntu    latest     5fb9b602c9b1  5 minutes ago     199MB
nginx           latest     341bf0f3ce6c  2 weeks ago       240MB
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ |
```

Typical result (approx):

Image Type	Size
nginx:latest	240 MB
nginx-ubuntu	199 MB
nginx-alpine	16 MB

Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```



```

aaroji26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
341bf0f3ce6c   2 weeks ago     CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     STOPIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     EXPOSE map[80/tcp:{}]                          0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENTRYPOINT ["/docker-entrypoint.sh"]           0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY docker-entrypoint.sh / # buildkit         8.19kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     RUN /bin/sh -c set -x && groupadd --syst...    86.6MB    buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV DYNPKG_RELEASE=1~trixie                     0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV PKG_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV ACME_VERSION=0.3.1                         0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_RELEASE=1~trixie                       0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_VERSION=0.9.5                          0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NGINX_VERSION=1.29.5                       0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB    debuerreotype 0.17
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
5fb9b602c9b1   6 minutes ago   CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      6 minutes ago   EXPOSE [80/tcp]                                0B        buildkit.dockerfile.v0
<missing>      6 minutes ago   RUN /bin/sh -c apt-get update && apt-get...    58.6MB    buildkit.dockerfile.v0
<missing>      9 days ago     /bin/sh -c #(nop) CMD ["/bin/bash"]            0B        buildkit.dockerfile.v0
<missing>      9 days ago     /bin/sh -c #(nop) ADD file:52c0e467fa2e92f10... 87.5MB    buildkit.dockerfile.v0
<missing>      9 days ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B        buildkit.dockerfile.v0
<missing>      9 days ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B        buildkit.dockerfile.v0
<missing>      9 days ago     /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH     0B        buildkit.dockerfile.v0
<missing>      9 days ago     /bin/sh -c #(nop) ARG RELEASE                  0B        buildkit.dockerfile.v0
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
f9f17069187b   2 minutes ago   CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      2 minutes ago   EXPOSE [80/tcp]                                0B        buildkit.dockerfile.v0
<missing>      2 minutes ago   RUN /bin/sh -c apk add --no-cache nginx # bu... 2.2MB     buildkit.dockerfile.v0
<missing>      3 weeks ago     CMD ["/bin/sh"]                                0B        buildkit.dockerfile.v0
<missing>      3 weeks ago     ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /... 9.11MB    buildkit.dockerfile.v0
aaroji26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ |

```

Observations:

- Ubuntu has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Tasks Using NGINX

Task 1: Serve Custom HTML Page

```

mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html

```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ mkdir html
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine$ cd html
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine/html$ nano index.html
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/nginx-alpine/html$ |
```



Run:

```
docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:~$ docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
09eb8b8716b5891dc9333fc143cde50dc7e4104a829dc685bcaa99341a1a4368
aarohi26@LAPTOP-4MMN5FCL:~$ |
```

Task 2: Reverse Proxy (Conceptual)

NGINX can:

- Forward traffic to backend services
- Load balance multiple containers
- Terminate SSL

Example use cases:

- Frontend for microservices
- API gateway
- Static file server

Part 6: Comparison Summary

Image Comparison Table

Feature	Official NGINX	Ubuntu + NGINX	Alpine + NGINX
Image Size	Medium	Large	Very Small
Ease of Use	Very Easy	Medium	Medium
Startup Time	Fast	Slow	Very Fast
Debugging Tools	Limited	Excellent	Minimal
Security Surface	Medium	Large	Small
Production Ready	Yes	Rarely	Yes

Part 7: When to Use What

Use Official NGINX Image When:

- Production deployment
- Standard web hosting
- Reverse proxy / load balancer

Use Ubuntu-Based Image When:

- Learning Linux + NGINX internals
- Heavy debugging needed
- Custom system-level dependencies

Use Alpine-Based Image When:

- Microservices
- CI/CD pipelines
- Cloud and Kubernetes workloads

Containerization_and_DevOps

Experiment 4: Docker Essentials

Dockerfile .dockerignore tagging publishing

Containerizing Applications with Dockerfile

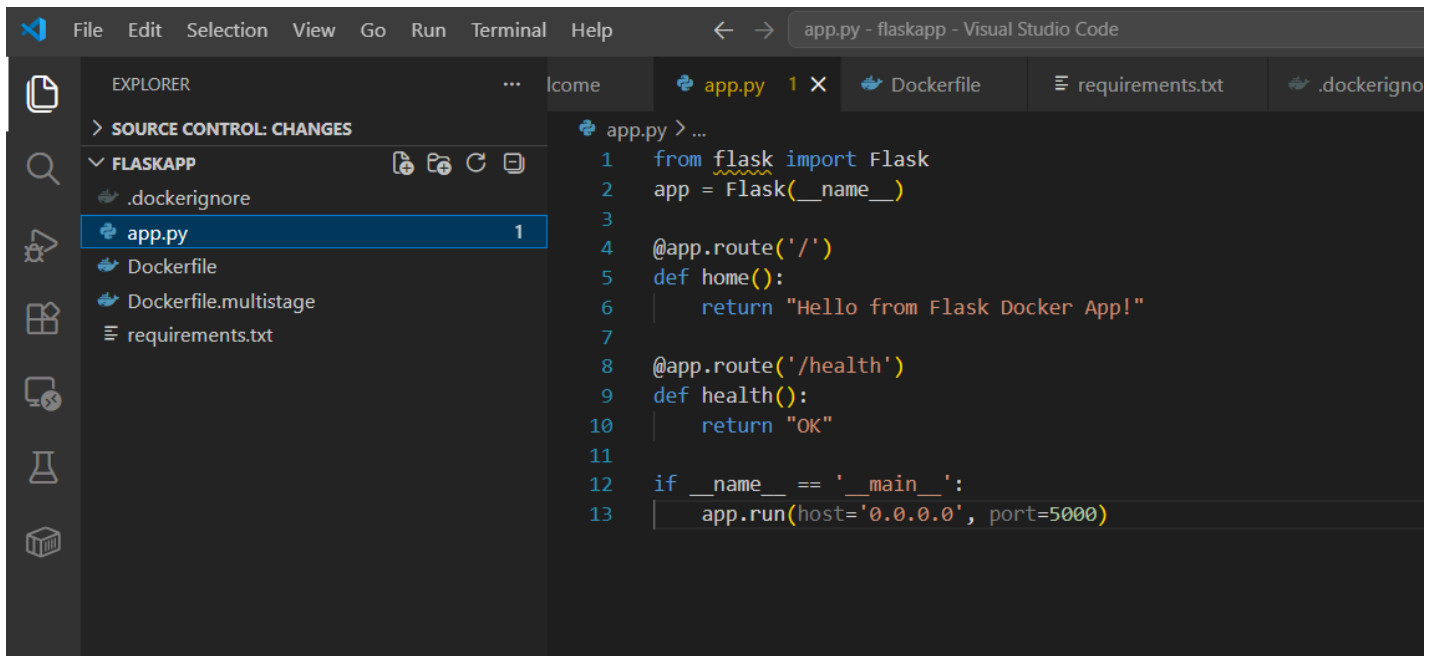
Step-1: Create a Directory for Python application

```
mkdir my-flask-app  
cd my-flask-app
```

```
C:\Users\ASUS>wsl  
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir flaskapp  
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ cd flaskapp
```

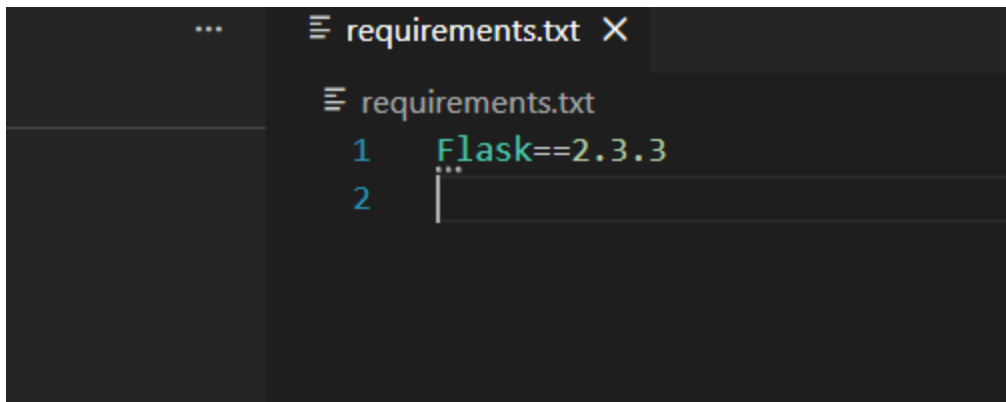
Step-2:- Create an **** app.py ****

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"  
  
@app.route('/health')  
def health():  
    return "OK"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```



Step-3:- Create **** requirements.txt ****

Flask==2.3.3



Step-4: Create ****Dockerfile****

```

# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

```

```
# Copy application code
```

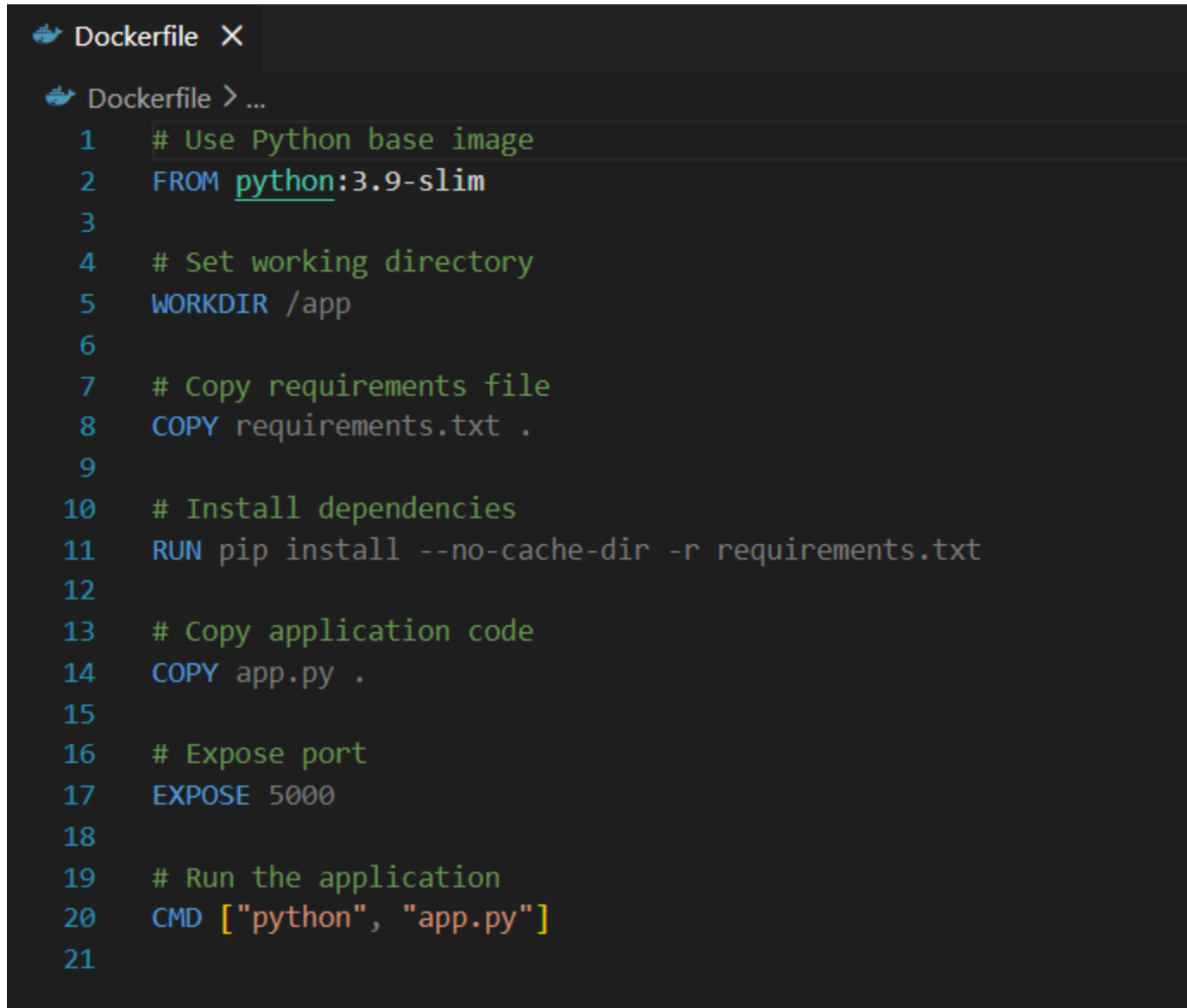
```
COPY app.py .
```

```
# Expose port
```

```
EXPOSE 5000
```

```
# Run the application
```

```
CMD ["python", "app.py"]
```



```
Dockerfile X
Dockerfile > ...
1  # Use Python base image
2  FROM python:3.9-slim
3
4  # Set working directory
5  WORKDIR /app
6
7  # Copy requirements file
8  COPY requirements.txt .
9
10 # Install dependencies
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Copy application code
14 COPY app.py .
15
16 # Expose port
17 EXPOSE 5000
18
19 # Run the application
20 CMD ["python", "app.py"]
21
```

Step-5:- Create a **** .dockerignore ****

```
# Python files
```

```
__pycache__/
```

```
*.pyc
```

```
*.pyo
```

```
*.pyd
```

Environment files

.env
.venv
env/
venv/

IDE files

.vscode/
.idea/

Git files

.git/
.gitignore

OS files

.DS_Store
Thumbs.db

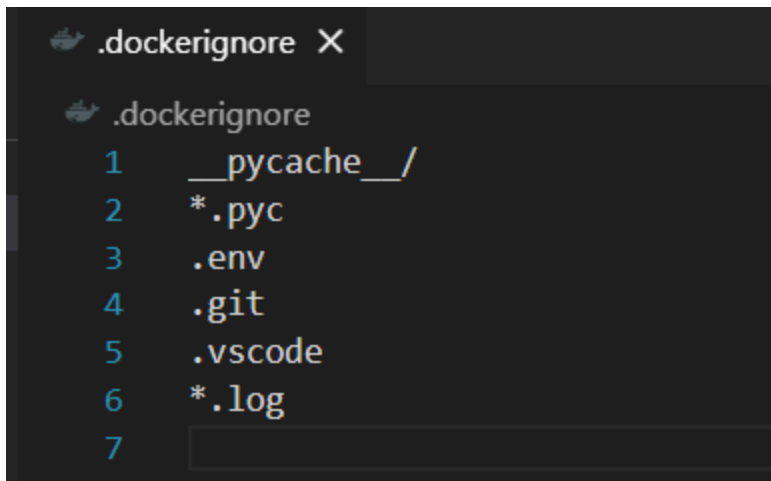
Logs

*.log
logs/

Test files

tests/
test_*.py

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ touch .dockerignore
```



Why *.dockerignore* is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -t flaskapp .
[+] Building 14.6s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => sha256:ea56f685404adf81680322f152d2cfe62115b30dda481c2c450078315beb508 251B / 251B
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08
=> => extracting sha256:ea56f685404adf81680322f152d2cfe62115b30dda481c2c450078315beb508
=> [internal] load build context
=> => transferring context: 342B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1c224d666ef4a83f20f3c5842181baaf1d56d7db77160a7022ad587d6986563d
=> => exporting config sha256:2df0c236b6e4c74c8ada5525b79e0c3ad1ee0bd23d034a23390c9f3024454640
=> => exporting attestation manifest sha256:610cb19896fd82c4704a3dcff20ddb1bbe6bbb3f1c3c607ad784a29e7e0b96c3
=> => exporting manifest list sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6f76c4b
=> => naming to docker.io/library/flaskapp:latest
=> => unpacking to docker.io/library/flaskapp:latest
```

Step-7:- Verify Image

```
docker images
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
flaskapp             latest       d05c6222eb8d     43 seconds ago  200MB
containerassgn-mybackend latest       c4739b79801f     2 days ago      123MB
containerassgn-mydatabase latest       8e67228ab04f     2 days ago      392MB
projectassignment-backend latest       c8b9f1cdc720     2 days ago      199MB
```

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -t flaskapp:1.0 .
docker tag flaskapp:latest flaskapp:v1.0
[+] Building 2.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1c224d666ef4a83f20f3c5842181baaf1d56d7db77160a7022ad587d6986563d
=> => exporting config sha256:2df0c236b6e4c74c8ada5525b79e0c3ad1ee0bd23d034a23390c9f3024454640
=> => exporting attestation manifest sha256:6eb59ae04c1f806e473fff86d53204ac22e5d5fdc73c872cd31bc7b18c403c60
=> => exporting manifest list sha256:978afe0cc550cf6a71b0ea813d9fad93dd4b0deebe9784020ed1b2681a5e2ecb
=> => naming to docker.io/library/flaskapp:1.0
=> => unpacking to docker.io/library/flaskapp:1.0
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build -t aarohi26/flaskapp:1.0 .
[+] Building 0.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load .dockerignore
=> => transferring context: 93B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --only=production
=> CACHED [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e8cc5b63d64bc1ddd1ffa5203ebb21bfa58cf44af39be6c4ceede3132fc108fc
=> => exporting config sha256:2c71dcc281f7fb7b39847e817ce1d6d6e723e46a13b87fe30bad1c00b919d575
=> => exporting attestation manifest sha256:e5acb8210abd371bcc05f68bdafcb22b05030650bd9f68e8230a4a24337e0147
=> => exporting manifest list sha256:1d265ba80b1bcff8919bce5c8299578bf7e471033214fd916aa92df0998cd613
=> => naming to docker.io/aarohi26/flaskapp:1.0
=> => unpacking to docker.io/aarohi26/flaskapp:1.0
```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker tag flaskapp:latest aarohi26/flaskapp:1.0
docker tag flaskapp:latest aarohi26/flaskapp:latest
```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker tag flaskapp:latest aarohi26/flaskapp:1.0
docker tag flaskapp:latest aarohi26/flaskapp:latest
```

Step-12: View Image Details

```
docker images
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
d05c6222eb8d	25 hours ago	CMD ["python" "app.py"]	0B	buildkit.dockerfile.v0

Step-13:- Show image history

```
docker history my-flask-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker inspect flaskapp
[
  {
    "Architecture": "amd64",
    "Author": "",
    "Comment": "buildkit.dockerfile.v0",
    "Config": {
      "ArgsEscaped": true,
      "Cmd": [
        "python",
        "app.py"
      ],
      "Entrypoint": null,
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Labels": null,
      "OnBuild": null,
      "User": "",
      "Volumes": null,
      "WorkingDir": "/app"
    },
    "Created": "2026-04-04T03:12:31.713516257Z",
    "Descriptor": {
      "digest": "sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b",
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "size": 856
    },
    "DockerVersion": "",
    "GraphDriver": {
      "Data": null,
      "Name": "overlayfs"
    }
  }
]
```

Step-14:- Inspect image details

```
docker inspect my-flask-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker inspect flaskapp
[
  {
    "Architecture": "amd64",
    "Author": "",
    "Comment": "buildkit.dockerfile.v0",
    "Config": {
      "ArgsEscaped": true,
      "Cmd": [
        "python",
        "app.py"
      ],
      "Entrypoint": null,
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Labels": null,
      "OnBuild": null,
      "User": "",
      "Volumes": null,
      "WorkingDir": "/app"
    },
    "Created": "2026-04-04T03:12:31.713516257Z",
    "Descriptor": {
      "digest": "sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b",
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "size": 856
    },
    "DockerVersion": "",
    "GraphDriver": {
      "Data": null,
      "Name": "overlayfs"
    }
  }
]
```

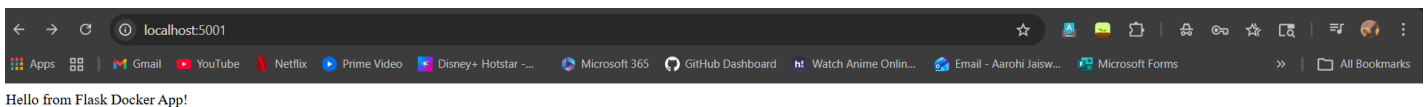
Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ curl http://localhost:5001
Hello from Flask Docker App!aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$
```

Step-16:- Test the application

```
curl http://localhost:5000
```



Step-17:- View running containers

```
docker ps
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker logs flask-container
docker stop flask-container
docker start flask-container
docker rm -f flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [04/Apr/2026 03:30:27] "GET / HTTP/1.1" 200 -
flask-container
flask-container
flask-container
```

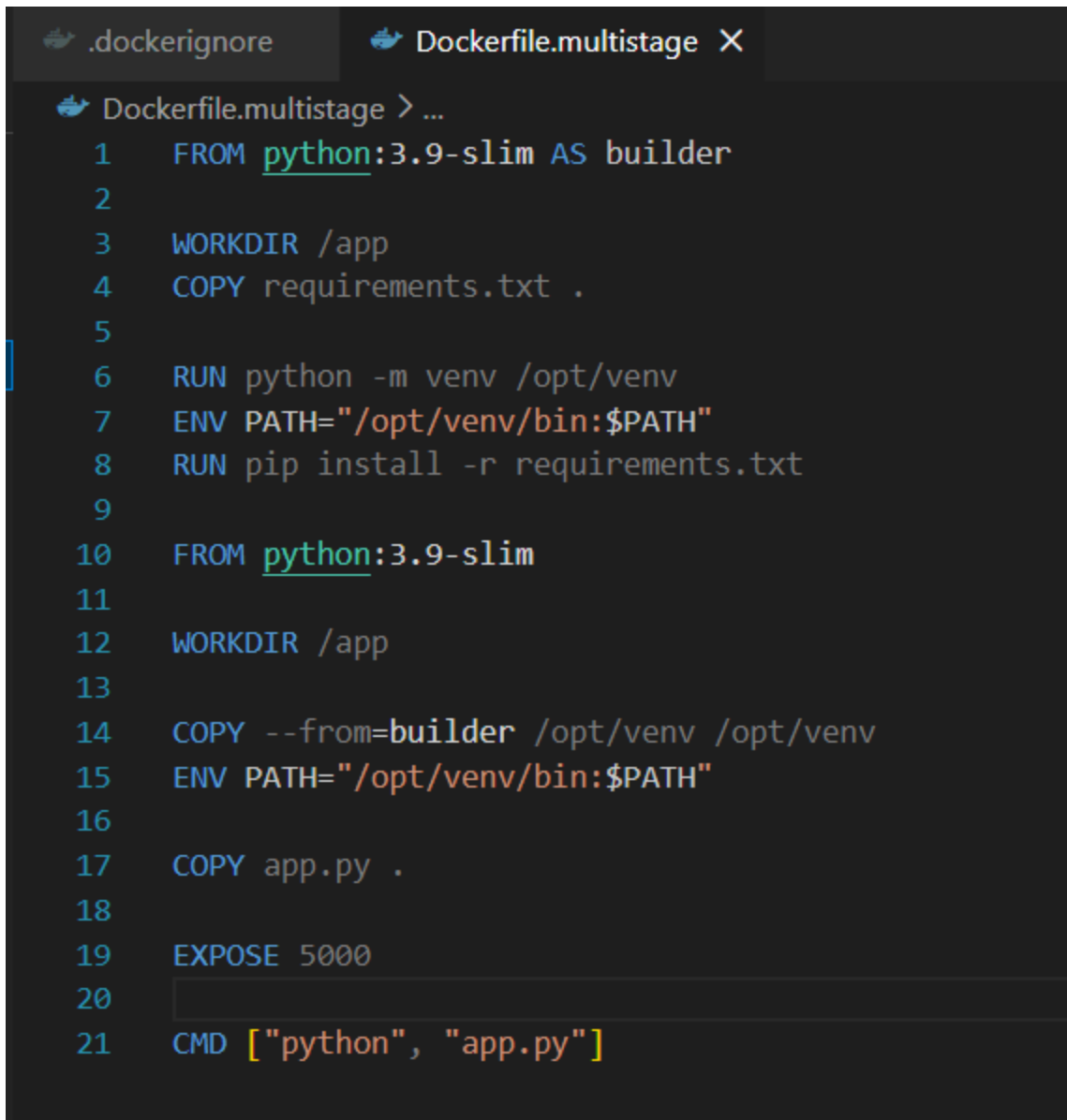
Step-18:- View container logs

```
docker logs flask-container
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ touch Dockerfile.multistage
```

Step-19:- Stop container

```
docker stop flask-container
```



The screenshot shows a code editor with two tabs: `.dockerignore` and `Dockerfile.multistage`. The `Dockerfile.multistage` tab is active, displaying a multi-stage Dockerfile. The file is numbered 1 through 21. The first stage (lines 1-9) is named `builder` and uses `python:3.9-slim` as the base image. It sets `WORKDIR /app`, copies `requirements.txt`, and runs `python -m venv /opt/venv`, `ENV PATH="/opt/venv/bin:$PATH"`, and `RUN pip install -r requirements.txt`. The second stage (lines 10-21) also uses `python:3.9-slim` as the base image. It sets `WORKDIR /app`, copies the virtual environment from the `builder` stage (`COPY --from=builder /opt/venv /opt/venv`), sets `ENV PATH="/opt/venv/bin:$PATH"`, copies `app.py`, exposes port 5000, and sets the command to `python app.py`.

```
.dockerignore Dockerfile.multistage X
Dockerfile.multistage > ...
1 FROM python:3.9-slim AS builder
2
3 WORKDIR /app
4 COPY requirements.txt .
5
6 RUN python -m venv /opt/venv
7 ENV PATH="/opt/venv/bin:$PATH"
8 RUN pip install -r requirements.txt
9
10 FROM python:3.9-slim
11
12 WORKDIR /app
13
14 COPY --from=builder /opt/venv /opt/venv
15 ENV PATH="/opt/venv/bin:$PATH"
16
17 COPY app.py .
18
19 EXPOSE 5000
20
21 CMD ["python", "app.py"]
```

Step-20:- Start stopped container

```
docker start flask-container
```



```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -t flaskapp-regular .
docker build -f Dockerfile.multistage -t flaskapp-optimized .
[+] Building 1.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1c224d666ef4a83f20f3c5842181baaf1d56d7db77160a7022ad587d6986563d
=> => exporting config sha256:2df0c236b6e4c74c8ada5525b79e0c3ad1ee8bd23d034a23390c9f3024454648
=> => exporting attestation manifest sha256:ccb44d69ee3508fbf57d9cc8b8bd47ad95e4ae49131eb491c1b11379a20f96aa1
=> => exporting manifest list sha256:cddc6afccf44a6dfdf560f41ef7a2bf80bb8597760f530fe8449b9771532c6c0
=> => naming to docker.io/library/flaskapp-regular:latest
=> => unpacking to docker.io/library/flaskapp-regular:latest
[+] Building 11.8s (12/12) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> => transferring dockerfile: 398B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt
=> [builder 4/5] RUN python -m venv /opt/venv
=> [builder 5/5] RUN pip install -r requirements.txt
=> [stage-1 3/4] COPY --from=builder /opt/venv /opt/venv
=> [stage-1 4/4] COPY app.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:82a48dddbc309115d20aa3589b1431c79f914ee0c2d43ddac6ff580f44439709
=> => exporting config sha256:8f4a58beae55e87d0f3485f1cd643bc6d8764be9e70a6e0e874541e7f1d7beca
=> => exporting attestation manifest sha256:2a065b6d8e7749b3e146b18e0f3375cf830595f60747b85917faf2cb869da5ff
=> => exporting manifest list sha256:93c15db1fc034b435626c52a5c466b60de4940c7df7cb570a961af3c1b049d5f
=> => naming to docker.io/library/flaskapp-optimized:latest
=> => unpacking to docker.io/library/flaskapp-optimized:latest

```

Step-21:- Remove container

```
docker rm flask-container
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 2.9s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> => transferring dockerfile: 398B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 89B
=> [internal] load build context
=> => transferring context: 63B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt
=> CACHED [builder 4/5] RUN python -m venv /opt/venv
=> CACHED [builder 5/5] RUN pip install -r requirements.txt
=> CACHED [stage-1 3/4] COPY --from=builder /opt/venv /opt/venv
=> CACHED [stage-1 4/4] COPY app.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:82a48dddbc309115d20aa3589b1431c79f914ee0c2d43ddac6ff580f44439709
=> => exporting config sha256:8f4a58beae55e87d0f3485f1cd643bc6d8764be9e70a6e0e874541e7f1d7beca
=> => exporting attestation manifest sha256:f8467946677a872e05755a1fab3be05e23e361e291a7c72c5dfc7a16fc6096d7
=> => exporting manifest list sha256:alice7d64801cf00cbc1f2e6d84be791243dcfb4a23150b880cbbe39dddeb2732
=> => naming to docker.io/library/flask-multistage:latest
=> => unpacking to docker.io/library/flask-multistage:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$

```

Step-22:- Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker images | grep flask-
flask-multistage      latest      a1ce7d64801c   25 hours ago   219MB
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple **** Dockerfile.multistage **:***

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
```

EXPOSE 5000

Run application

CMD ["python", "app.py"]

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker login
Authenticating with existing credentials... [Username: aarohi26]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

Step-24:- Build regular image

`docker build -t flask-regular .`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker tag flaskapp:latest aarohi26/flaskapp:1.0
docker tag flaskapp:latest aarohi26/flaskapp:latest
```

Step-25:- Build multi-stage image

`docker build -f Dockerfile.multistage -t flask-multistage .`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker push aarohi26/flaskapp:1.0
docker push aarohi26/flaskapp:latest
The push refers to repository [docker.io/aarohi26/flaskapp]
cc36eeef8762: Pushed
65d5eacb340e: Pushed
c7dfc3a6ec32: Pushed
38513bd72563: Pushed
b3ec39b36ae8: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
e144e59d3cce: Pushed
a8d60ef6ec86: Pushed
1.0: digest: sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b size: 856
The push refers to repository [docker.io/aarohi26/flaskapp]
65d5eacb340e: Layer already exists
c7dfc3a6ec32: Layer already exists
a8d60ef6ec86: Layer already exists
ea56f685404a: Layer already exists
e144e59d3cce: Layer already exists
cc36eeef8762: Already exists
b3ec39b36ae8: Layer already exists
fc7443084902: Layer already exists
38513bd72563: Layer already exists
latest: digest: sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b size: 856
```

Step-26:- Compare sizes

`docker images | grep flask-`

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker pull aaroHi26/flaskapp:latest
latest: Pulling from aaroHi26/flaskapp
Digest: sha256:d05c6222eb8dc920e26d1f1b8c66eb46711ca66f2142c5488dca6bf6ffc76c4b
Status: Image is up to date for aaroHi26/flaskapp:latest
docker.io/aaroHi26/flaskapp:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ docker run -d -p 5000:5000 aaroHi26/flaskapp:latest
84832f1953727d5755d953c25b3f38d6545662a1d5c85995fa9126a6afd49765
```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp$ mkdir mynodeapp
cd mynodeapp
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-28:- Tag image for Docker Hub

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano app.js
```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

```
aarohi26@LAPTOP-4MMN5FCL x + v
GNU nano 7.2
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

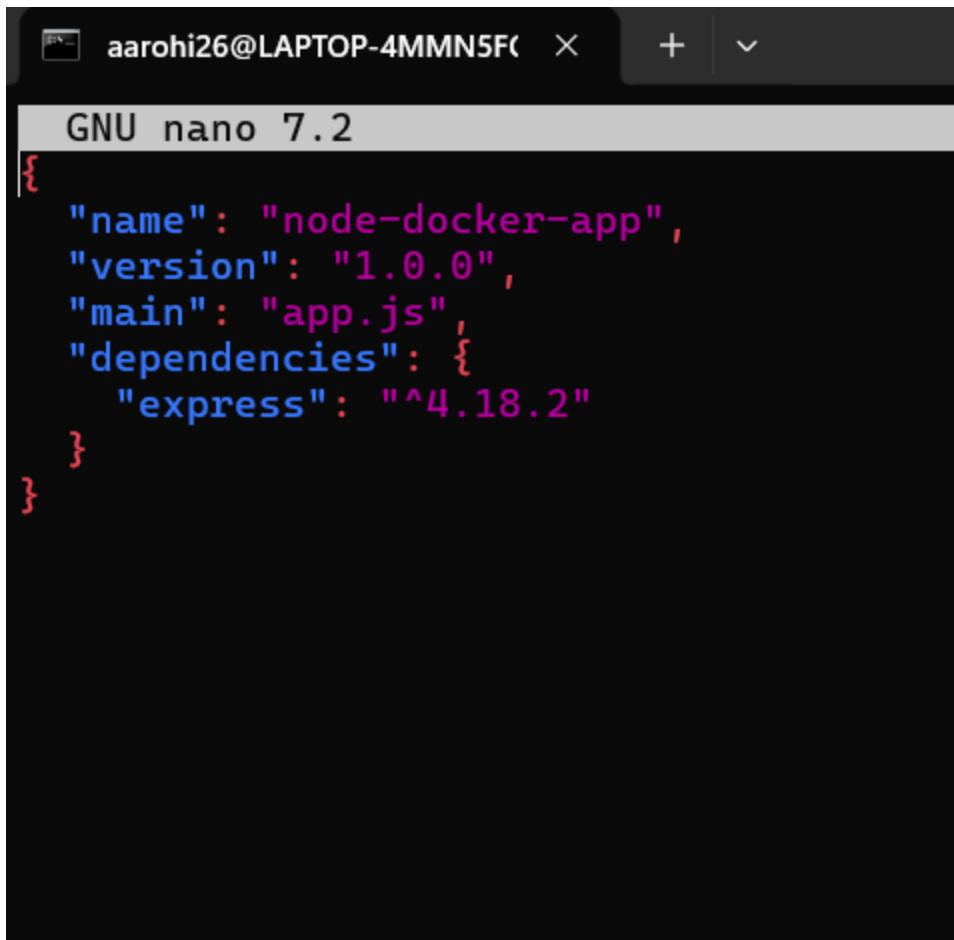
Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano package.json
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-31:- Pull from Docker Hub (on another machine)

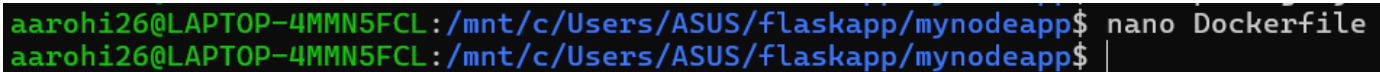
```
docker pull username/my-flask-app:latest
```



```
aarohi26@LAPTOP-4MMN5FC  ×  +  ∨  
GNU nano 7.2  
{  
  "name": "node-docker-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

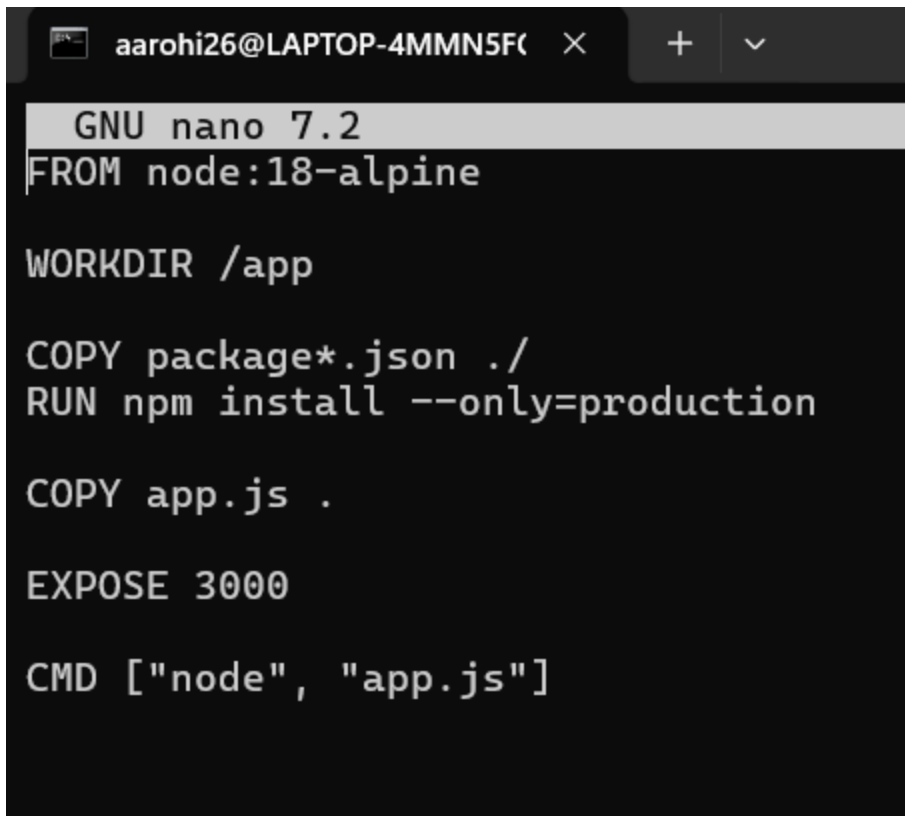


```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano Dockerfile  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$
```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```
mkdir my-node-app  
cd my-node-app
```



The screenshot shows a terminal window with the title 'aaroji26@LAPTOP-4MMN5F6'. The terminal is running 'GNU nano 7.2' and displays the following Dockerfile content:

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step-34:- Create a **app.js** :

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```



```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build -t mynodeapp .
[+] Building 20.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 514B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b88df5a2e0549 446B / 446B
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feal61b2b5a3d0e 40.01MB / 40.01MB
=> => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feal61b2b5a3d0e
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b88df5a2e0549
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --only=production
=> [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1a73b71a08505c6cc8a10e35ea87f562b902ff6eaa5890423faebe09d598ce36
=> => exporting config sha256:1d5339ee8e3e31d34e4480921b72606a8324cf71bb31d208f968eaa47f75bab
=> => exporting attestation manifest sha256:cbda2857cc39f5b4975ee93a51ca84eeb678b0c21ec3f2d11656818c3cbc1ea6
=> => exporting manifest list sha256:1a24a03a6ef47e3eca5e0d2b10befe5c3524e7ce43e4a95a9fbbe4594104af89
=> => naming to docker.io/library/mynodeapp:latest
=> => unpacking to docker.io/library/mynodeapp:latest
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |

```

Step-35:- Create a **** package.json ****

```

{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker run -d -p 3001:3000 --name mynodecontainer mynodeapp
5375d72462d5ba43c642be39d0a5407bc4c9db31ebbf3ef06a20c6530670b50a
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |

```

Step-36:- Create a Node.js Dockerfile

```

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ curl http://localhost:3001
Hello from Node.js Docker!aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |

```

Step-37:- Build image

```
docker build -t my-node-app .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build -t myapp:latest -t myapp:v2.0 -t aarohi26/myapp:production .
[+] Building 18.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --only=production
=> CACHED [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:1a73b71a08505c6cc8a10e35ea87f562b902ff6eaa5890423faebe09d598ce36
=> => exporting config sha256:1d5339ee8e3e31d34e4480921b72606a8324cf71bb31d208f968eeaa47f75bab
=> => exporting attestation manifest sha256:f1d0e519b386ecd6e7f0fc430e994912d57c5f74a6e3f8b79386c647757a57e6
=> => exporting manifest list sha256:bd03bd4fc08f26677db46fcc497c55b861a6d4dfdc1795dcb3dc7df696a77651
=> => naming to docker.io/library/myapp:latest
=> => unpacking to docker.io/library/myapp:latest
=> => naming to docker.io/library/myapp:v2.0
=> => unpacking to docker.io/library/myapp:v2.0
=> => naming to docker.io/aarohi26/myapp:production
=> => unpacking to docker.io/aarohi26/myapp:production
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-38:- Run container

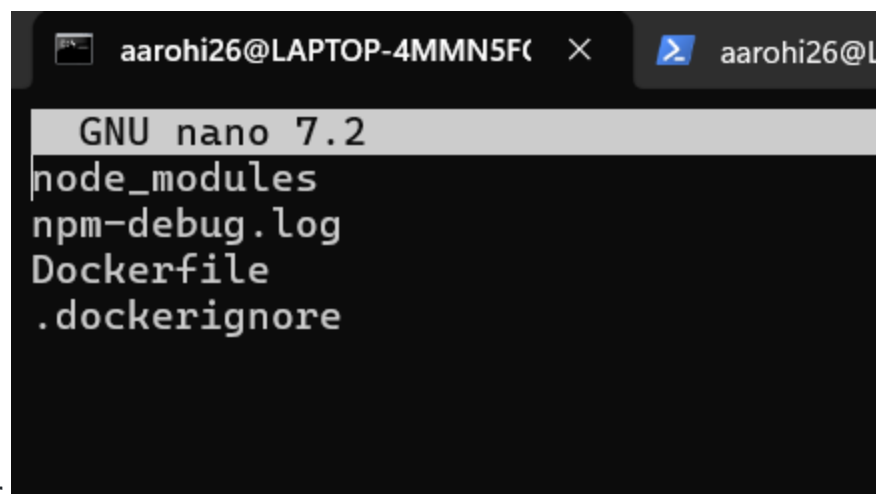
```
docker run -d -p 3000:3000 --name node-container my-node-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano Dockerfile2
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```

Step-39:- Test on Terminal

```
curl http://localhost:3000
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ nano .dockerignore
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ |
```



Step-40:- Test on Browser

DockerHub

View Pushed Image on [DockerHub](#)

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/flaskapp/mynodeapp$ docker build --no-cache -t clean-app .
[+] Building 7.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 93B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --only=production
=> [5/5] COPY app.js .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e8cc5b63d64bc1ddd1ffa5203ebb21bfa58cf44af39be6c4ceede3132fc108fc
=> => exporting config sha256:2c71dcc281f7fb7b39847e817ce1d6d6e723e46a13b87fe30bad1c00b919d575
=> => exporting attestation manifest sha256:7747b88e6c319382e3395be059332c9c05d84fef948411aee523e53f43e2153a
=> => exporting manifest list sha256:9e307f6a588267720353b86a37d96b44728644b3bdf150d22af7180f888b5da
=> => naming to docker.io/library/clean-app:latest
=> => unpacking to docker.io/library/clean-app:latest
=> => unpacking to docker.io/library/clean-app:latest
```

Conclusion

Essential Docker Commands Cheatsheet

Command	Purpose	Example
docker build	Build image	docker build -t myapp .
docker run	Run container	docker run -p 3000:3000 myapp
docker ps	List containers	docker ps -a
docker images	List images	docker images
docker tag	Tag image	docker tag myapp:latest myapp:v1
docker login	Login to Dockerhub using username and password or token	echo "token" docker login -u username --password-stdin
docker push	Push to registry	docker push username/myapp
docker pull	Pull from registry	docker pull username/myapp

Command	Purpose	Example
<code>docker rm</code>	Remove container	<code>docker rm container-name</code>
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Common Workflow Summary

Development Workflow:

```
# 1. Create Dockerfile and .dockerignore
# 2. Build image
docker build -t myapp .

# 3. Test Locally
docker run -p 8080:8080 myapp

# 4. Tag for production
docker tag myapp:latest myapp:v1.0

# 5. Push to registry
docker push myapp:v1.0
```

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0

# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image
2. **.dockerignore** excludes unnecessary files (important for security and performance)
3. **Tagging** helps version and organize images
4. **Multi-stage builds** create smaller, more secure production images
5. **Docker Hub** allows sharing and distributing images
6. **Always test** images locally before publishing

Cleanup

```
# Remove all stopped containers  
docker container prune
```

```
# Remove unused images  
docker image prune
```

```
# Remove everything unused  
docker system prune -a
```

[!\[\]\(830769b31eeeaca920791081939ff8ba_img.jpg\) Previous](#) [!\[\]\(198f559926258ddfad814817bda0ffbc_img.jpg\) Home](#)

Containerization_and_DevOps

Experiment 5:- Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -it --name test-container ubuntu /bin/bash
root@9ed18ed4baab:/# echo "Hello World" > /data/message.txt
```

```
echo "Hello World" > /data/message.txt
cat /data/message.txt
exit
```

```
root@9ed18ed4baab:/# mkdir /data
echo "Hello World" > /data/message.txt
exit
exit
```

```
docker start test-container
docker exec test-container cat /data/message.txt
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker start test-container
docker exec test-container cat /data/message.txt
test-container
Hello World
```

Solution: Docker Volumes

Lab 2: Volume Types

Anonymous Volume

```
docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1 | grep -A 5 Mounts
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1 | grep -A 5 Mounts
12756ef2028d9036838c76454b9b7abd4522ceb121207bd6f43a2db1425b77d2
DRIVER      VOLUME NAME
local      assignment1_pgdata
local      caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b
local      containerassgn_pgdata
local      containerzatiassgn_postgres_data
local      dockerassignment_postgres_data
local      minikube
local      mydata
local      myvolume
local      myvolumej
local      project_pg_data
local      projectassignment_pgdata
"Mounts": [
  {
    "Type": "volume",
    "Name": "caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b",
    "Source": "/var/lib/docker/volumes/caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b/_data",
    "Destination": "/app/data",
    "Driver": "local",
    "Mode": "rw"
  }
]
```

Named Volume

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
```



```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
mydata
9bbc1e429fee92065e1a89bb01ebed6ad5231ef4b8ad5ac1dcc15db4f2e63729
DRIVER      VOLUME NAME
local       assignment1_pgdata
local       caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b
local       containerassgn_pgdata
local       containerzatiassgn_postgres_data
local       dockerassignment_postgres_data
local       minikube
local       mydata
local       myvolume
local       myvolumej
local       project_pg_data
local       projectassignment_pgdata
[
  {
    "CreatedAt": "2026-03-20T21:05:30Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |

```

Bind Mount

```

mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
0ef975ef0bf4f9834c394c680df077976e8bc07f0e8d848894ffe738e0b0bae4
From Host

```

Lab 3: Practical Examples

MySQL Persistent Storage

```
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
60c3210ab1f6: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
c3b8cfb10b41: Pull complete
440208b3f29f: Pull complete
5915b0e5580c: Pull complete
6375ec02106f: Pull complete
771773a1fe7a: Pull complete
1378718ee973: Pull complete
d4c7048d1cf1: Pull complete
de0490608ca2: Download complete
2f1dcce71f1f: Download complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd000
Status: Downloaded newer image for mysql:8.0
aa5cc1cbd019cce39b4c1f0868acdb9e1d2541fcc6cd0b6967c7ab2d3cd97186
```

```
docker stop mysql-db
docker rm mysql-db
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker stop mysql-db
docker rm mysql-db
mysql-db
mysql-db
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
7833f4de09d84fa230a8ca16b4fdaf1102152aaadf5a7d021522a620d4327ca6
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

NGINX Config Mount

```
mkdir ~/nginx-config
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir ~/nginx-config
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
echo 'server {
    listen 80;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ echo 'server {
    listen 80;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d \
--name nginx-custom \
-p 8080:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name nginx-custom \
-p 8080:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
e5f71609dafe98d4f078773fee69f3a22848867038bc8d4a9559e819b70ebf45
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
curl http://localhost:8080
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ curl http://localhost:8080
Hello from mounted config!aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Lab 4: Volume Commands

```
docker volume ls
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume ls
DRIVER      VOLUME NAME
local       assignment1_pgdata
local       caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b
local       containerassgn_pgdata
local       containerzatiassgn_postgres_data
local       dockerassignment_postgres_data
local       minikube
local       mydata
local       mysql-data
local       myvolume
local       myvolumej
local       project_pg_data
local       projectassignment_pgdata
```

```
docker volume create app-volume
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume create app-volume
app-volume
```

```
docker volume inspect app-volume
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-06T18:33:21Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
```

docker volume prune

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume rm volume-name
Error response from daemon: volume volume-name not found
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume rm app-volume
```

docker volume **rm** volume-name

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume rm app-volume
app-volume
```

docker **cp** local-file.txt container-name:/path/in/volume

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker cp local-file.txt test-container:/data
Successfully copied 2.05kB to test-container:/data
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Part 2: Environment Variables

Setting Variables

```
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  my-node-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name app1 \
-e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
-e DEBUG="true" \
ubuntu sleep 300
339cbfdbaa127e66fa7de6b160d87b292e941ce2b208437b452c20b4fbb9c330
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Using .env File

```
echo "DATABASE_HOST=localhost" > .env
echo "API_KEY=secret123" >> .env
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ echo "DATABASE_HOST=localhost" > .env
echo "API_KEY=secret123" >> .env
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d --env-file .env my-app
```

Dockerfile Example

```
ENV NODE_ENV=production
ENV PORT=3000
```

Flask Example

```
import os
from flask import Flask

app = Flask(__name__)

@app.route('/config')
def config():
    return {
        "db": os.environ.get("DATABASE_HOST"),
        "debug": os.environ.get("DEBUG")
    }

app.run(host="0.0.0.0", port=5000)
```

Test Variables

```
docker run -d \  
  --name flask-app \  
  -p 5000:5000 \  
  -e DEBUG=true \  
  flask-app
```

```
docker exec flask-app env  
curl http://localhost:5000/config
```

Part 3: Docker Monitoring

```
docker stats  
docker stats --no-stream  
  
docker top container-name  
  
docker logs -f container-name  
  
docker inspect container-name  
  
docker events
```

Simple Monitoring Script

```
#!/bin/bash  
docker ps  
docker stats --no-stream  
docker system df
```

Part 4: Docker Networks

List Networks

```
docker network ls
```

Bridge Network

```
docker network create my-network
```

```
docker run -d --name web1 --network my-network nginx
```

```
docker run -d --name web2 --network my-network nginx
```

```
docker exec web1 curl http://web2
```

Host Network

```
docker run -d --network host nginx
```

None Network

```
docker run -d --network none alpine sleep 3600
```

Network Commands

```
docker network create app-network
```

```
docker network connect app-network container-name
```

```
docker network disconnect app-network container-name
```

```
docker network rm app-network
```

Multi-Container Example

```
docker network create app-network
```

```
docker run -d \  
  --name postgres-db \  
  --network app-network \  
  -e POSTGRES_PASSWORD=secret \  
  postgres:15
```



```
docker run -d \  
  --name web-app \  
  --network app-network \  
  -p 8080:3000 \  
  -e DATABASE_HOST="postgres-db" \  
  node-app
```

Debugging

```
docker network inspect bridge  
docker exec container-name ping google.com  
docker port container-name
```

Part 5: Real-World Example

```
docker network create myapp-network
```

```
docker run -d \  
  --name postgres \  
  --network myapp-network \  
  -e POSTGRES_PASSWORD=pass \  
  postgres:15
```

```
docker run -d \  
  --name redis \  
  --network myapp-network \  
  redis:7-alpine
```

```
docker run -d \  
  --name flask-app \  
  --network myapp-network \  
  -p 5000:5000 \  
  flask-app:latest
```

Cheatsheet

```
docker volume create <name>
docker run -v <volume>:/path

docker run -e VAR=value

docker stats
docker logs -f <container>

docker network create <name>
```

Practice

- Create DB with volume
- Multi-service setup
- Analyze logs
- Network isolation

Cleanup

```
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
docker volume prune -f
docker network prune -f
docker image prune -f
```

Key Takeaways

- Volumes store persistent data
- Env variables control config
- Monitoring helps debugging
- Networks connect containers
- Use .env for security

Containerization_and_DevOps



Docker & Docker Compose Lab README



Overview

This lab demonstrates:

- Difference between **Docker Run (imperative)** and **Docker Compose (declarative)**
- Running **single-container** and **multi-container** applications
- Converting Docker commands into Compose files
- Using **Dockerfile with Compose**
- Advanced concepts like **multi-stage builds** and **resource limits**

◆ PART A – Docker Run vs Docker Compose

Docker Run (Imperative)

```
docker run -d \  
  --name lab-nginx \  
  -p 8081:80 \  
  -v $(pwd)/html:/usr/share/nginx/html \  
  nginx:alpine
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d --name lab-nginx -p 8081:80 -v $(pwd)/html:/usr/share/nginx/html nginx:alpine  
Unable to find image 'nginx:alpine' locally  
alpine: Pulling from library/nginx  
612c0c1df4c5: Pull complete  
453da7dbc73e: Pull complete  
4a8b0b2a5b19: Pull complete  
781ff50d2644: Pull complete  
583599bb7d38: Pull complete  
82736a35d0e7: Pull complete  
6a0ac1617861: Pull complete  
aee4e54b3865: Pull complete  
e8fc446e336c: Download complete  
6192e1e6a438: Download complete  
Digest: sha256:8aa63af009a39ecd6c28d61da578a5447378c10bb097a069e3a3e0fb42bb6b19  
Status: Downloaded newer image for nginx:alpine  
2788759c517aee34245512b77f7cb315b15e3ac5cf249cd24e7ccf08d3e182c8  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Docker Compose (Declarative)

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Run:

```
docker compose up -d
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker compose up -d
WARN[0000] /mnt/c/Users/ASUS/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
✔Network asus_default Created 0.1s
✔Container my-nginx Created 0.1s
```

Stop:

```
docker compose down
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker compose down
WARN[0000] /mnt/c/Users/ASUS/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container my-nginx Removed 0.5s
✔Network asus_default Removed 0.3s
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ *
```

◆ PART B – PRACTICAL TASKS

Task 1: Single Container

Using Docker Run

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```

docker ps

```
Run 'docker run --help' for more information
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
2788759c517a   nginx:alpine   "/docker-entrypoint..." 12 minutes ago Up 12 minutes   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
lab-nginx
24f1d00fba3a   containerassgn-mybackend  "uvicorn main:app == "    2 weeks ago   Up 20 minutes   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
```

Access:

http://localhost:8081

Stop:

```
docker stop lab-nginx
docker rm lab-nginx
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker stop lab-nginx
lab-nginx
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker rm lab-nginx
lab-nginx
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Using Docker Compose

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ nano docker-compose.yml
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
GNU nano 7.2
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
```

Run:

```
docker compose up -d
docker compose ps
docker compose down
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker compose up -d
docker compose ps
docker compose down
WARN[0000] /mnt/c/Users/ASUS/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
✔Network asus_default Created 0.1s
✔Container my-nginx Created 0.2s
WARN[0000] /mnt/c/Users/ASUS/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
my-nginx nginx:alpine "/docker-entrypoint..." nginx Less than a second ago Up Less than a second 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
WARN[0000] /mnt/c/Users/ASUS/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container my-nginx Removed 0.7s
✔Network asus_default Removed 0.4s
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Task 2: Multi-Container (WordPress + MySQL)

Docker Run Approach

```
docker network create wp-net
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker network create wp-net
e7ea6a7d89519a24911674589015405b04ab38e2efd62f6137b1831c31047dfa
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d \  
  --name mysql \  
  --network wp-net \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  -e MYSQL_DATABASE=wordpress \  
  mysql:5.7
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \  
  --name mysql \  
  --network wp-net \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  -e MYSQL_DATABASE=wordpress \  
  mysql:5.7  
Unable to find image 'mysql:5.7' locally  
5.7: Pulling from library/mysql  
43d05e938198: Pull complete  
68c3898c2015: Pull complete  
e9f03a1c24ce: Pull complete  
6b95a940e7b6: Pull complete  
ffc89e9dfd88: Pull complete  
ae71319cb779: Pull complete  
1c56c3d4ce74: Pull complete  
064b2d298fba: Pull complete  
90986bb8de6e: Pull complete  
df9a4d85569b: Pull complete  
20e4dcae4c69: Pull complete  
96889c47a44a: Download complete  
9eb6a52b8441: Download complete  
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb  
Status: Downloaded newer image for mysql:5.7  
1b2eb08a13f6c88d96802e14cf6982a5e11e3ed62ad37a4bf1560c50175cb952  
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d \  
  --name wordpress \  
  --network wp-net \  
  -p 8082:80 \  
  -e WORDPRESS_DB_HOST=mysql \  
  -e WORDPRESS_DB_PASSWORD=secret \  
  wordpress:latest
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name wordpress \
--network wp-net \
-p 8082:80 \
-e WORDPRESS_DB_HOST=mysql \
-e WORDPRESS_DB_PASSWORD=secret \
wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
78a70974d738: Pull complete
5435b2dcd5c: Pull complete
0dbbd63ae63a: Pull complete
d5af5dfa43d4: Pull complete
e24121de25e6: Pull complete
4f4fb700ef54: Pull complete
568912d7e28d: Pull complete
3f3cb8cd0960: Pull complete
73ce216c5445: Pull complete
7d9f8a57d57a: Pull complete
de502dcf081a: Pull complete
5020203f7d33: Pull complete
b4b1305df06a: Pull complete
4443d9c71a47: Pull complete
271a7052ce75: Pull complete
9bde0c34d1f4: Pull complete
e052461f1b41: Pull complete
871c6a1a47cc: Pull complete
9cb746f747ab: Pull complete
ef17106e0cde: Pull complete
6562f7228ec3: Pull complete
79e11af4b256: Pull complete
90be0470639d: Pull complete
1fa182c1d458: Pull complete
05bb872ab5e8: Download complete
32a1d2f81bd2: Download complete
Digest: sha256:1a758de2f424f735a9f8ad181783a9ed29e028594869a3323903d92532b35102
Status: Downloaded newer image for wordpress:latest
755b2297cdfee64fab24f58ca0537c5b5db587ea25f053b8ec4dd5d5a888d279
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Docker Compose Approach

```
version: '3.8'
```

```
services:
```

```
mysql:
```

```
image: mysql:5.7
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD: secret
```

```
MYSQL_DATABASE: wordpress
```

```
volumes:
```

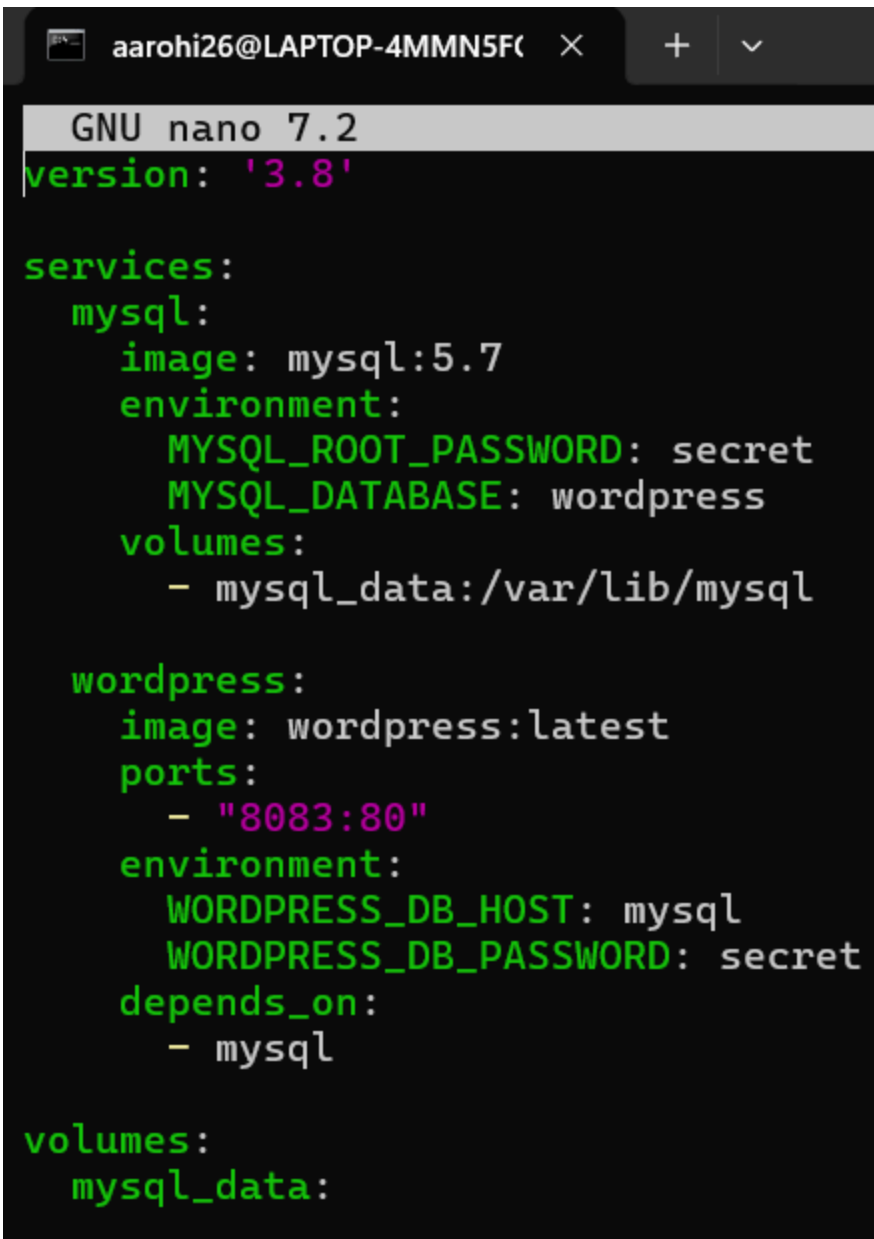
```
- mysql_data:/var/lib/mysql
```



```
wordpress:
  image: wordpress:latest
  ports:
    - "8082:80"
  environment:
    WORDPRESS_DB_HOST: mysql
    WORDPRESS_DB_PASSWORD: secret
  depends_on:
    - mysql

volumes:
  mysql_data:
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir wordpress-docker
cd wordpress-docker
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/wordpress-docker$ nano docker-compose.yml
```



```
GNU nano 7.2
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8083:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```

Run:

```
docker compose up -d
docker compose down -v
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/wordpress-docker$ docker compose up -d
WARN[0000] /mnt/c/Users/ASUS/wordpress-docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potenti
al confusion
[+] up 3/3
✔ Network wordpress-docker_default      Created      0.0s
✔ Container wordpress-docker-mysql-1    Created      0.1s
✔ Container wordpress-docker-wordpress-1 Created      0.2s

aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/wordpress-docker$ docker compose down -v
WARN[0000] /mnt/c/Users/ASUS/wordpress-docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potenti
al confusion
[+] down 4/4
✔ Container wordpress-docker-wordpress-1 Removed      1.6s
✔ Container wordpress-docker-mysql-1    Removed      1.4s
✔ Network wordpress-docker_default      Removed      0.4s
✔ Volume wordpress-docker_mysql_data    Removed      0.0s
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/wordpress-docker$
```

◆ PART C – CONVERSION TASKS

Task 3 – Problem 1 (Web App)

```
version: '3.8'

services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: false
    restart: unless-stopped
```

Task 3 – Problem 2 (Volume + Network)

```
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
```

```
volumes:
  - pgdata:/var/lib/postgresql/data
networks:
  - app-net
```

```
backend:
  image: python:3.11-slim
  container_name: backend
  ports:
    - "8000:8000"
  environment:
    DB_HOST: postgres-db
    DB_USER: admin
    DB_PASS: secret
  depends_on:
    - postgres-db
  networks:
    - app-net
```

```
volumes:
  pgdata:

networks:
  app-net:
```

Run:

```
docker compose up -d
docker compose down -v
```

Task 4 – Resource Limits

```
version: '3.8'

services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
```

`cpus: "0.5"``memory: 256M`

Explanation

- `deploy` works only in Docker Swarm
- In normal Compose → ignored
- Swarm → enforces CPU & memory limits

◆ PART D – DOCKERFILE WITH COMPOSE

Task 5 – Node App Build

app.js

```
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```

Dockerfile

```
FROM node:18-alpine

WORKDIR /app
COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

docker-compose.yml

```
version: '3.8'

services:
  nodeapp:
```

```
build:
  context: .
  dockerfile: Dockerfile
container_name: custom-node-app
ports:
  - "3000:3000"
```

Run:

```
docker compose up --build -d
```

image vs build

- image: → pulls prebuilt image
- build: → creates custom image from Dockerfile

Task 6 – Multi-Stage Dockerfile

Example (Node)

```
# Stage 1 - Build
FROM node:18-alpine as builder
WORKDIR /app
COPY . .
RUN npm install

# Stage 2 - Production
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app .

EXPOSE 3000
CMD ["node", "app.js"]
```

Compose

```
version: '3.8'

services:
```

```
app:
  build: .
  ports:
    - "3000:3000"
  environment:
    NODE_ENV: production
  volumes:
    - ./app
```

Check size:

docker images

◆ EXPERIMENT 6B – WORDPRESS PROJECT

docker-compose.yml

```
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
```

```
WORDPRESS_DB_PASSWORD: wppass
WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html
```

```
volumes:
  db_data:
  wp_data:
```

Run:

```
docker compose up -d
docker ps
```

Access:

<http://localhost:8080>

Stop:

```
docker compose down
```

◆ SCALING

```
docker compose up --scale wordpress=3
```

⚠ Issues:

- Port conflict
- No load balancing

Solution: Add Nginx reverse proxy

◆ DOCKER SWARM

```
docker swarm init
docker stack deploy -c docker-compose.yml wpstack
docker service scale wpstack_wordpress=3
```



FINAL LEARNING

✓ Docker Run → quick but messy ✓ Docker Compose → clean & scalable ✓ Dockerfile → custom builds ✓ Multi-stage → optimized images ✓ Swarm → production scaling



DONE

You now have:

- Single container setup
- Multi-container app
- Conversion skills
- Build-based deployment
- Scaling knowledge

Containerization_and_DevOps

Lab Experiment 7: CI/CD Pipeline using Jenkins, GitHub & Docker Hub

Aim

Design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

Objectives

- Understand CI/CD workflow using Jenkins (GUI-based automation server)
- Create a structured GitHub repository containing application code and a Jenkinsfile
- Build Docker images from source code automatically
- Securely store Docker Hub credentials inside Jenkins
- Automate build & push using GitHub Webhook triggers
- Use the same Docker host as the Jenkins agent

Tech Stack

Tool	Role
Jenkins	CI/CD automation server
GitHub	Source code + pipeline definition host
Docker	Build & package application
Docker Hub	Container image registry
Webhook	Trigger automation on code push

Workflow Overview

Developer → GitHub Push → Webhook → Jenkins → Docker Build → Docker Hub

Project Structure

```
my-app/  
├── app.py           # Flask web application  
├── requirements.txt # Python dependencies  
├── Dockerfile       # Image build instructions  
└── Jenkinsfile      # Pipeline definition
```

Setup Guide

Part A: GitHub Repository

1. Application Code (app.py)

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hello from CI/CD Pipeline!"  
  
app.run(host="0.0.0.0", port=80)
```

```
GNU nano 7.2
from flask import Flask
app = Flask(__name__)

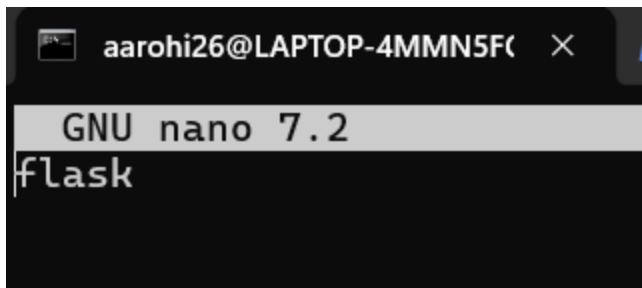
@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=80)
```

2.

Dependencies (requirements.txt)

```
flask
```



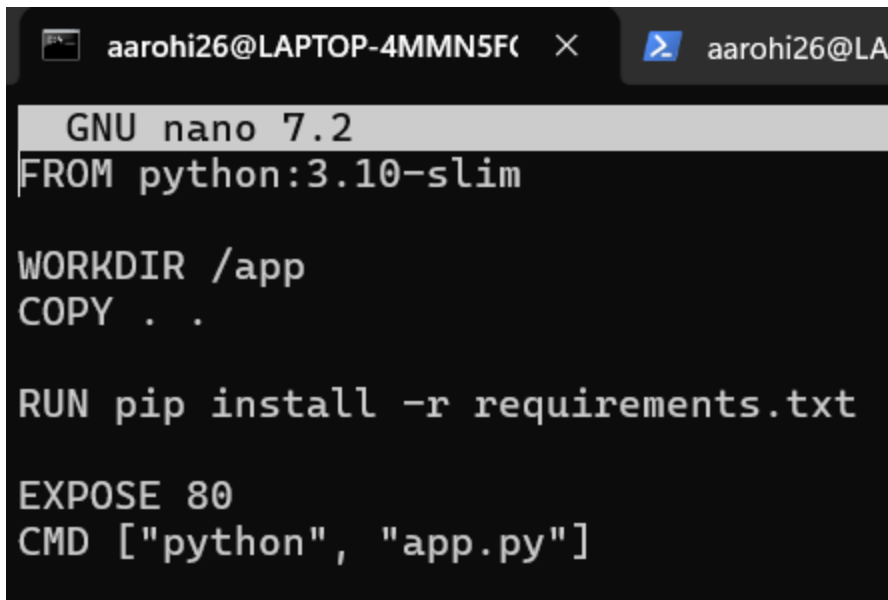
3. Dockerfile

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```



```

aaroHi26@LAPTOP-4MMN5F6  X  aaroHi26@LA
GNU nano 7.2
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]

```

4. Jenkinsfile

```

pipeline {
    agent any

    environment {
        IMAGE_NAME = "your-dockerhub-username/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/your-username/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
                }
            }
        }

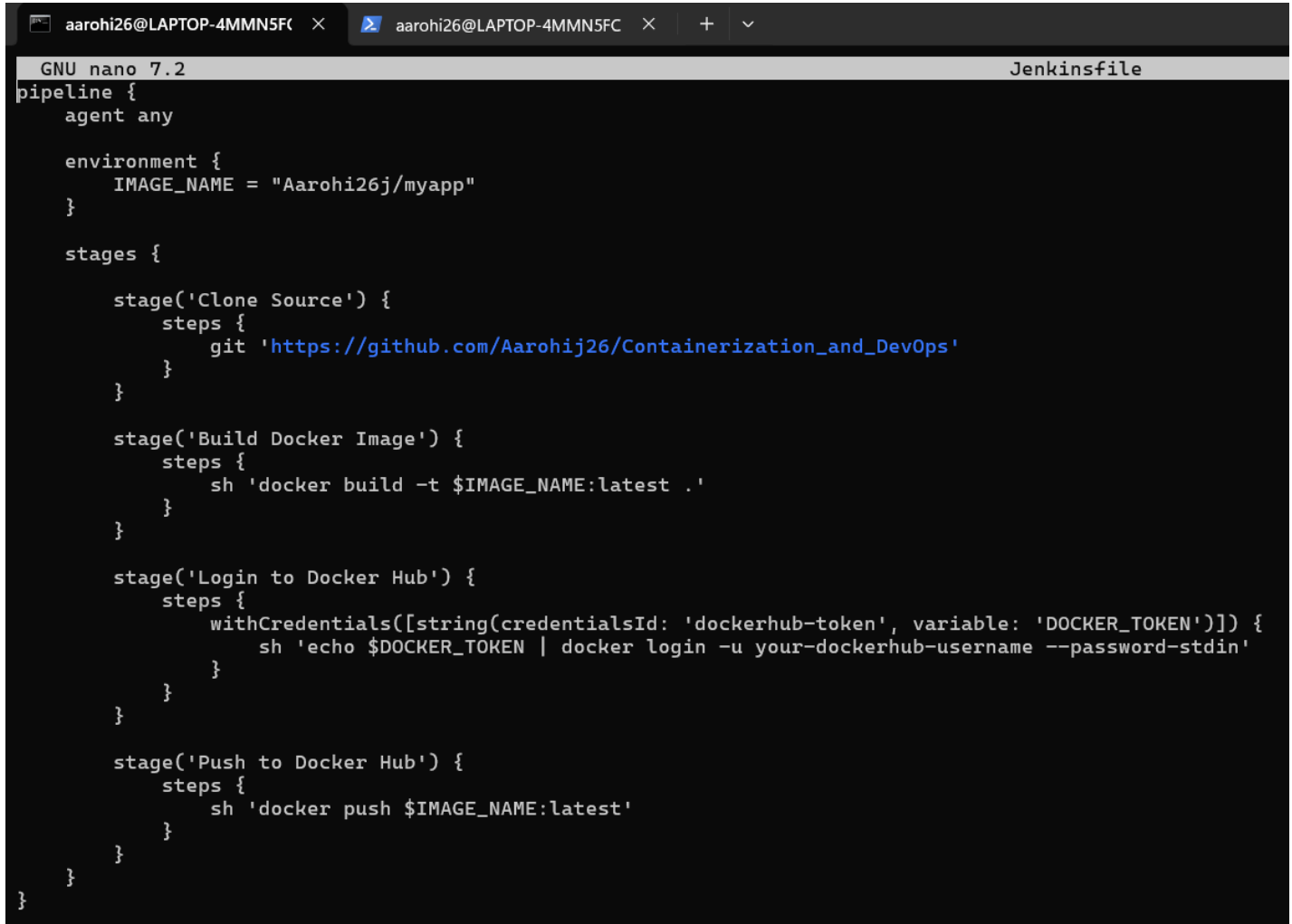
        stage('Push to Docker Hub') {

```

```

    steps {
        sh 'docker push $IMAGE_NAME:latest'
    }
}
}
}

```



The screenshot shows a terminal window with two tabs. The active tab is titled 'aaroji26@LAPTOP-4MMN5FC' and contains a nano editor window titled 'Jenkinsfile'. The editor shows a Jenkinsfile script with the following content:

```

GNU nano 7.2 Jenkinsfile
pipeline {
    agent any

    environment {
        IMAGE_NAME = "Aaroji26j/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/Aaroji26/Containerization_and_DevOps'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}

```

Part B: Jenkins Setup (via Docker Compose)

docker-compose.yml

```

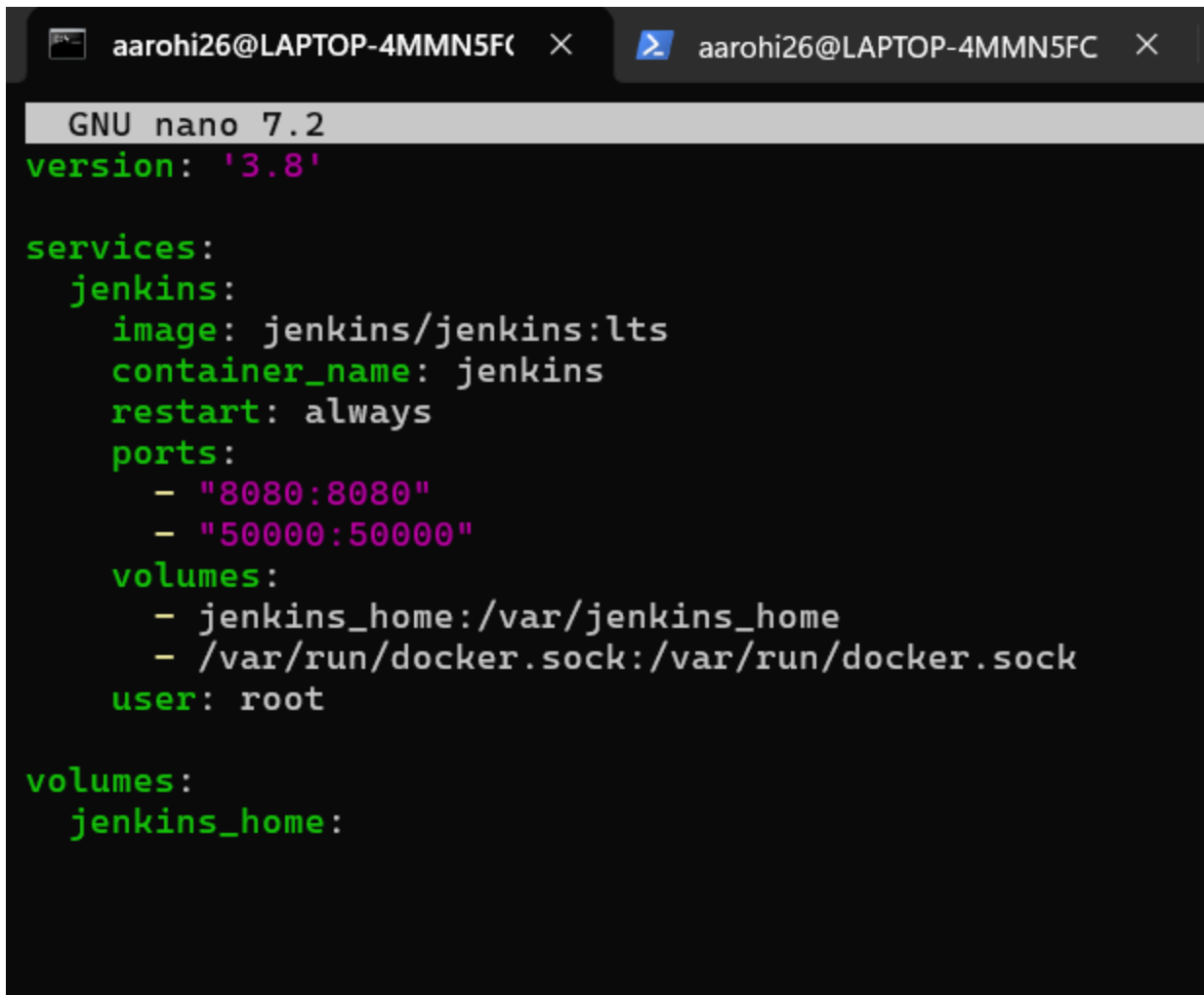
version: '3.8'

services:
    jenkins:
        image: jenkins/jenkins:lts
        container_name: jenkins
        restart: always
        ports:

```

```
- "8080:8080"
- "50000:50000"
volumes:
- jenkins_home:/var/jenkins_home
- /var/run/docker.sock:/var/run/docker.sock
user: root
```

```
volumes:
  jenkins_home:
```



```
aarohi26@LAPTOP-4MMN5FC x aarohi26@LAPTOP-4MMN5FC x
GNU nano 7.2
version: '3.8'

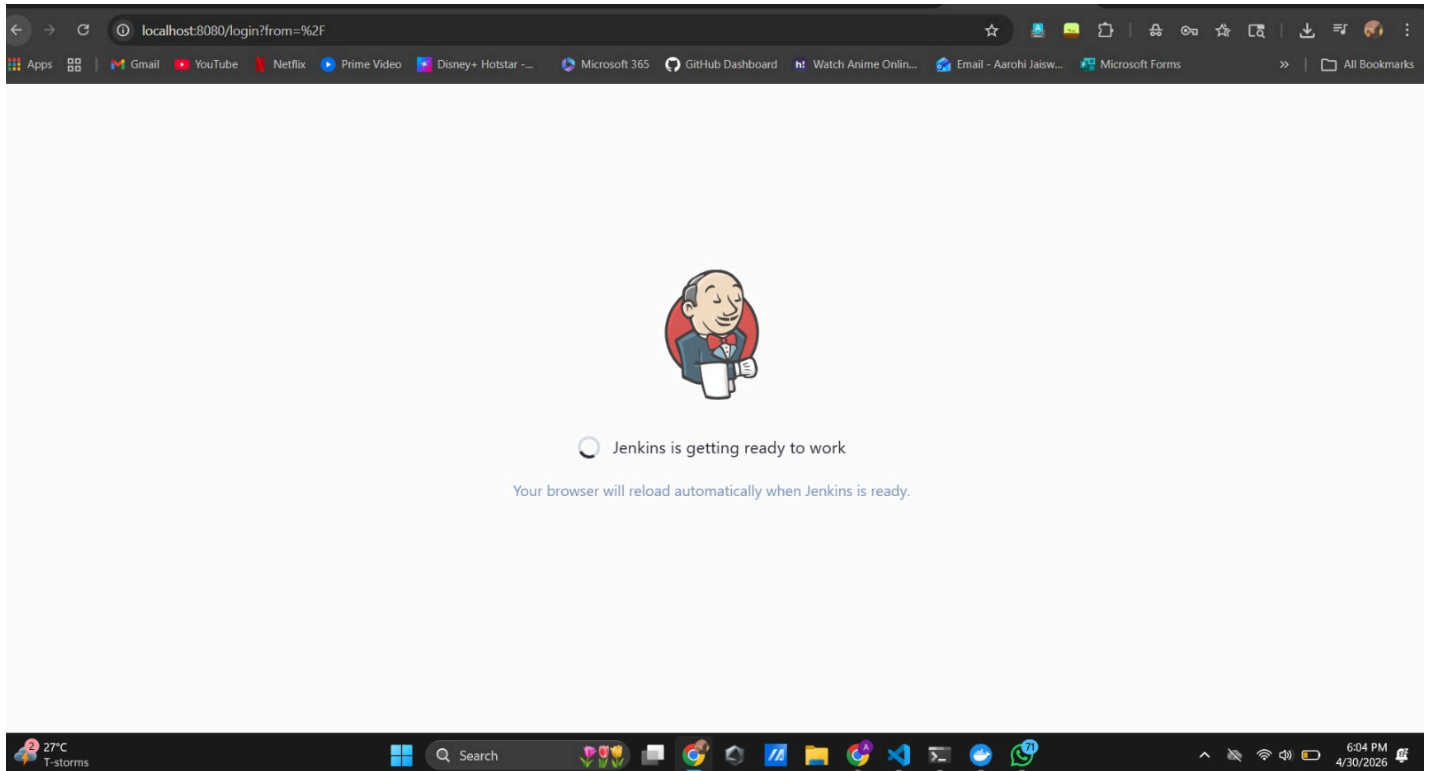
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

Start Jenkins:

```
docker-compose up -d
```

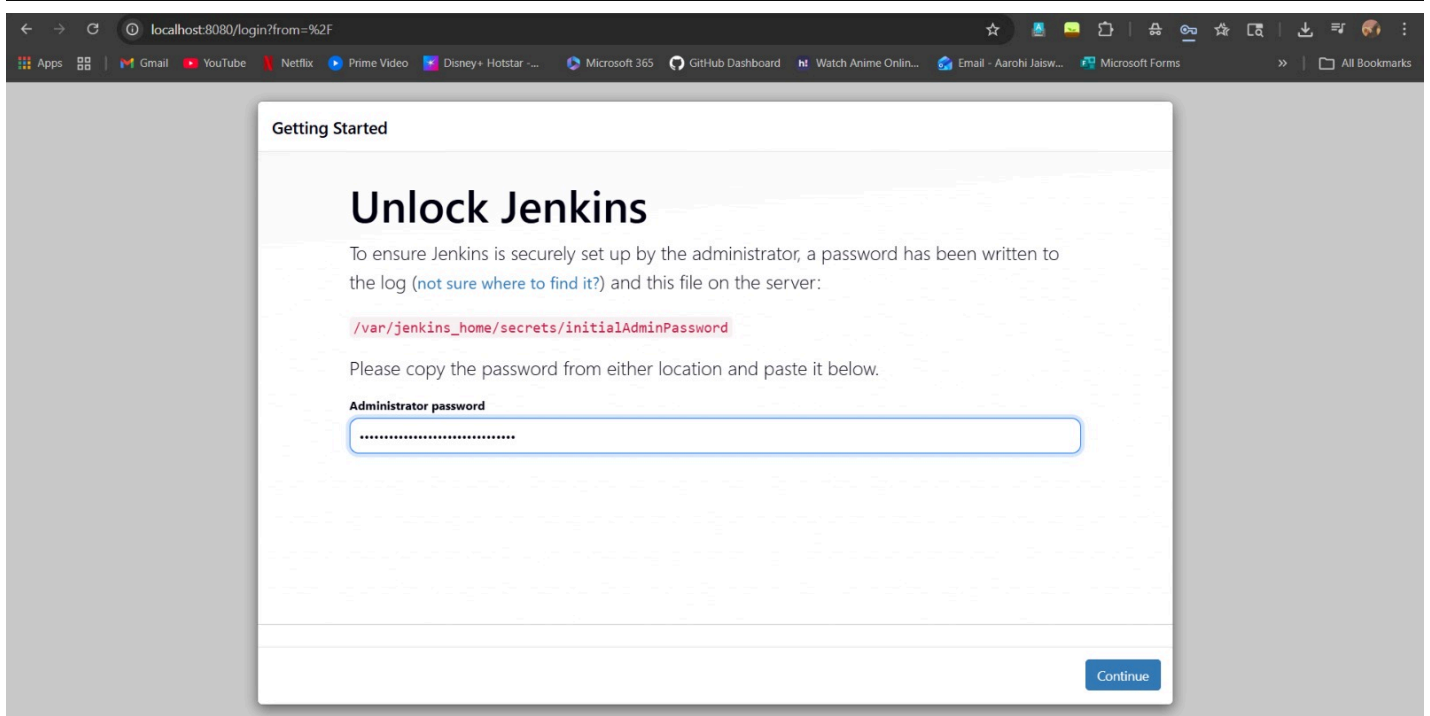
Access Jenkins: <http://localhost:8080>



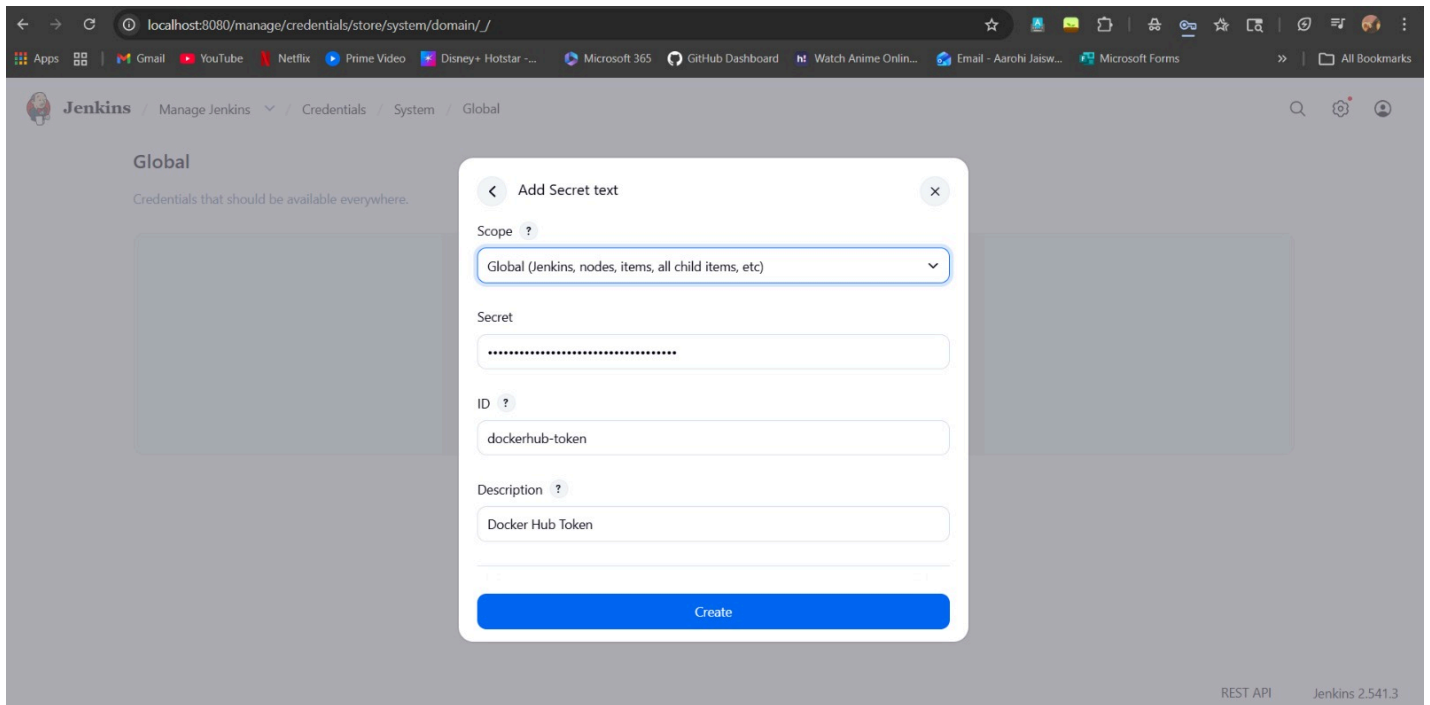
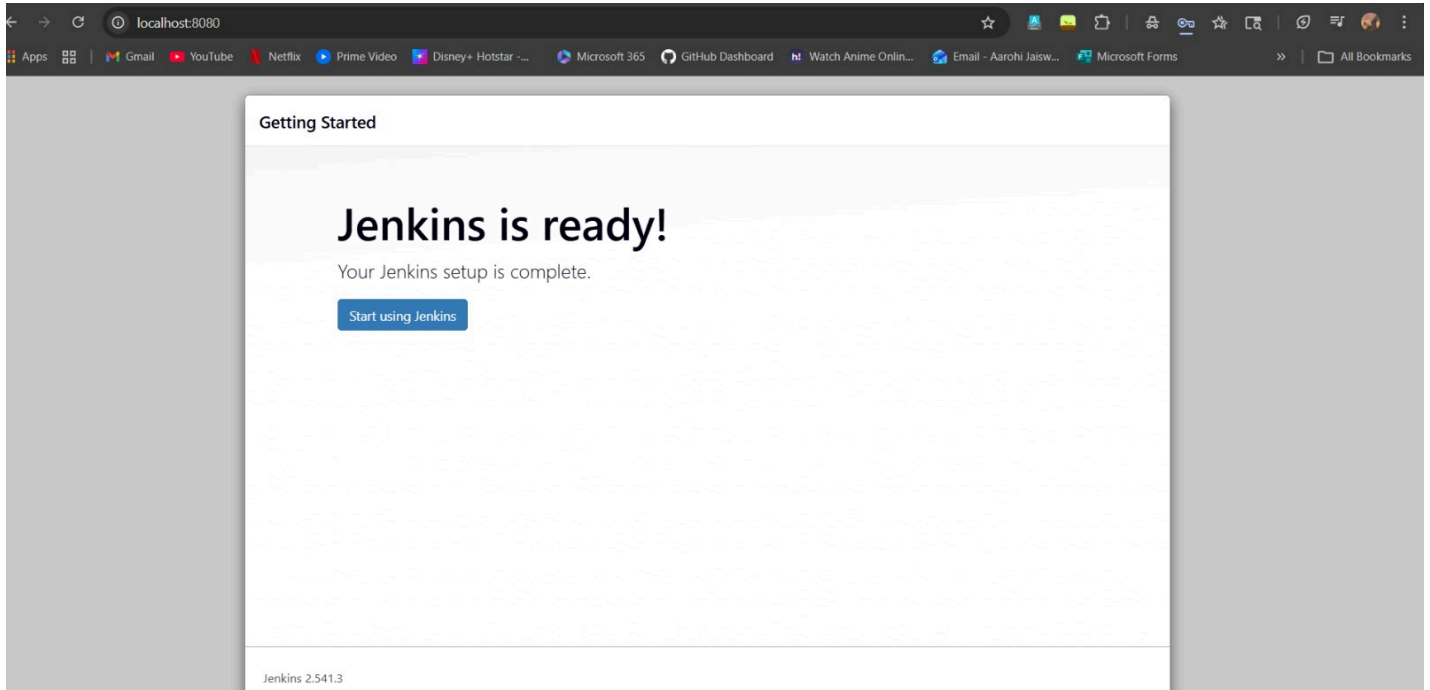
Get initial admin password:

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

```
aarohi26@LAPTOP-4MMN5FCL:~/cicd-app$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
787cde10ea2a45debaa283acfb44b3d2
```



After unlocking: install suggested plugins and create an admin user.



Part C: Jenkins Configuration

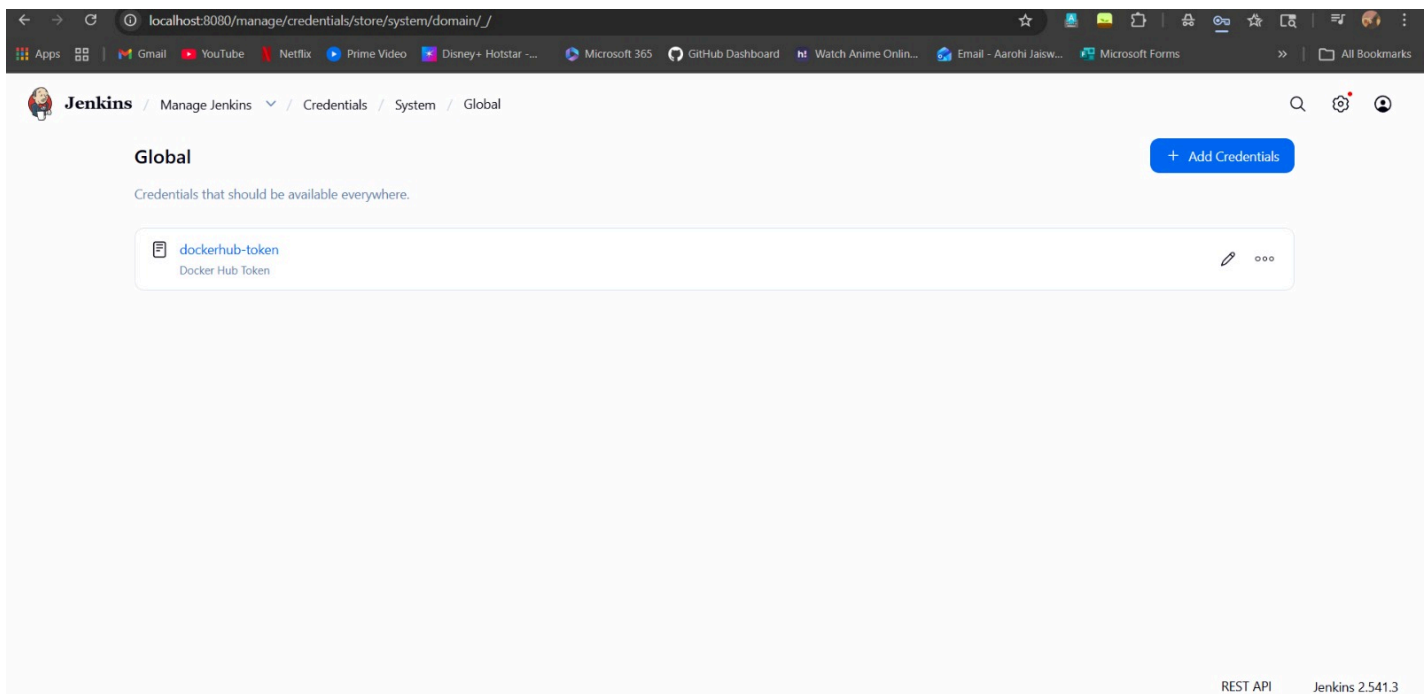
1. Add Docker Hub Credentials

Manage Jenkins → Credentials → Add Credentials

Type: Secret Text

ID: dockerhub-token

Value: <your Docker Hub Access Token>



2. Create Pipeline Job

New Item → Pipeline

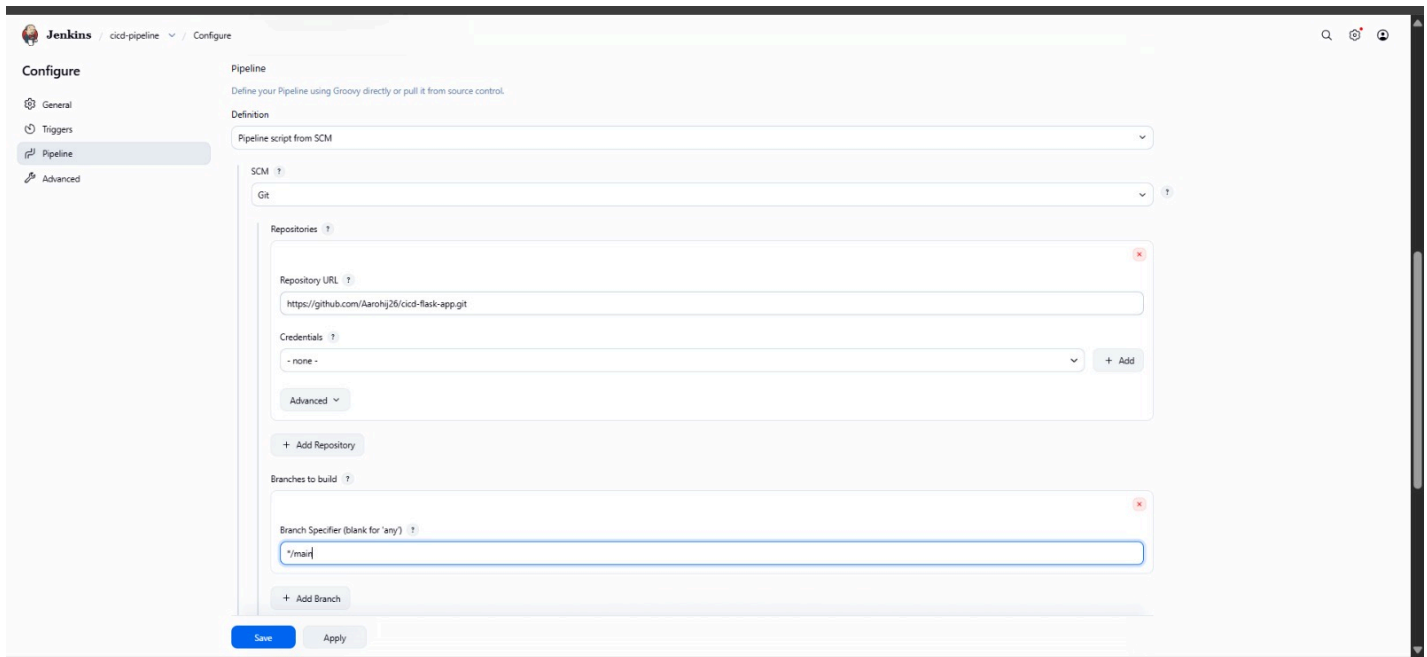
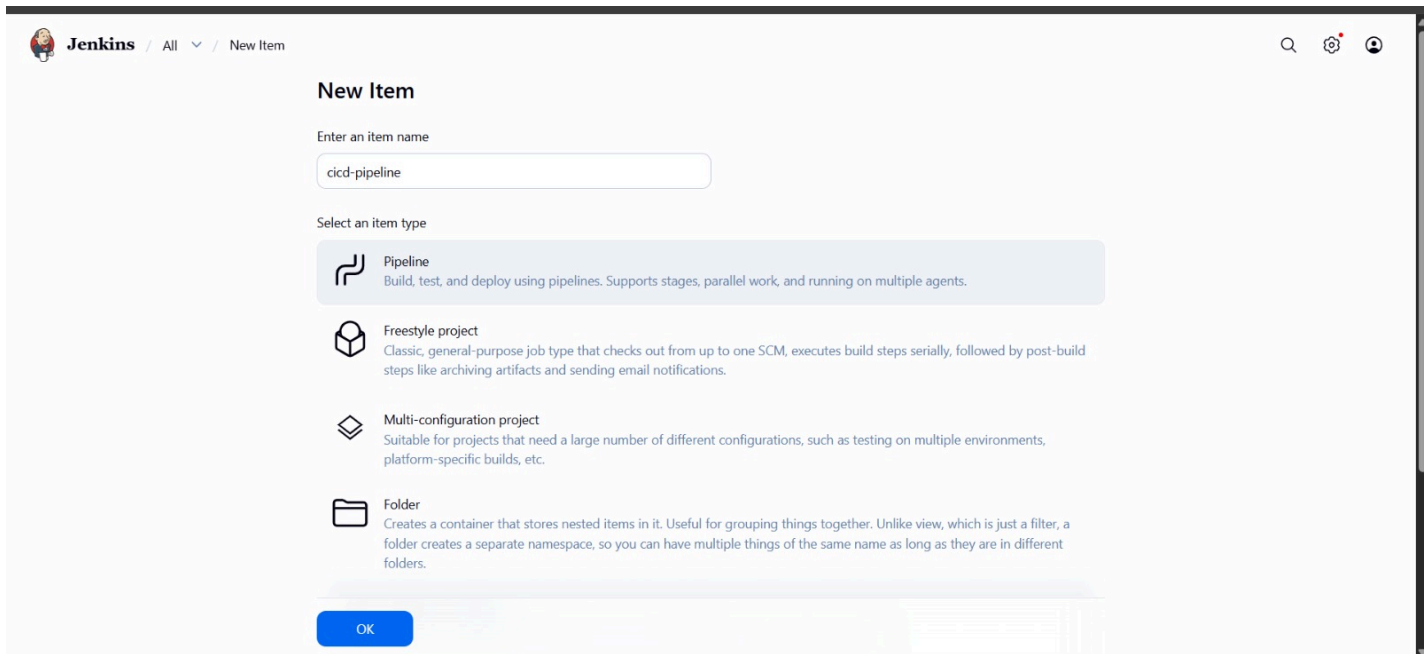
Name: ci-cd-pipeline

Pipeline script from SCM

SCM: Git

Repo URL: <https://github.com/your-username/my-app.git>

Script Path: Jenkinsfile



Part D: GitHub Webhook

GitHub Repo → Settings → Webhooks → Add Webhook

Payload URL: `http://<your-server-ip>:8080/github-webhook/`

Events: Push events

Pipeline Execution Flow

Stage	Action
Code Push	Developer pushes code to GitHub
Webhook Trigger	GitHub notifies Jenkins automatically
Clone	Jenkins pulls latest code
Build	Docker builds the image using <code>Dockerfile</code>
Auth	Jenkins injects Docker Hub token securely
Push	Image is pushed to Docker Hub
Done	Image available globally on Docker Hub

Understanding `withCredentials`

The `withCredentials` block securely injects stored secrets as temporary environment variables — the secret never appears in plain text in your code or logs.

```
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {  
    sh 'echo $DOCKER_TOKEN | docker login -u your-username --password-stdin'  
}
```

Part	Meaning
string	Type of secret (plain text token)
credentialsId	ID used when saving secret in Jenkins
variable	Temporary env variable name inside the block

 `$DOCKER_TOKEN` is only accessible **inside** the `withCredentials` block — it disappears after.

Never do this:

```
sh 'docker login -u user -p mypassword' //  Hardcoded secret
```

Key Notes

- Jenkins is GUI-based but pipelines are code-driven via `Jenkinsfile`
- Docker socket (`/var/run/docker.sock`) is mounted so Jenkins can control host Docker directly — no separate agent needed
- Always store secrets in Jenkins Credentials — never hardcode them
- Webhook makes the entire pipeline fully automatic on every push

Observations

- Jenkins GUI simplifies CI/CD pipeline management
- GitHub acts as both source code host and pipeline definition store
- Docker ensures consistent, reproducible builds
- Webhook eliminates the need for manual pipeline triggers

Result

Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline definition are maintained in GitHub
- Jenkins automatically detects changes via webhook
- Docker image is built on the host agent
- Image is securely pushed to Docker Hub

Viva Questions

1. What is the role of `Jenkinsfile` ?
2. How does Jenkins integrate with GitHub?
3. Why is Docker used in CI/CD?
4. What is a webhook and how does it work?
5. Why store the Docker Hub token in Jenkins credentials instead of the `Jenkinsfile` ?
6. What is the benefit of using the same host as the Jenkins agent?

Containerization_and_DevOps



Experiment 9: Ansible



Problem Statement

Managing infrastructure manually across multiple servers leads to configuration drift, inconsistent environments, and repetitive tasks. Scaling becomes difficult with manual SSH-based administration.



What is Ansible?

Ansible is an open-source automation tool used for configuration management, application deployment, and orchestration.

Features:

- Agentless (uses SSH)
- YAML-based playbooks
- Idempotent (safe to run multiple times)
- Push-based architecture



Key Concepts

Component Description Control Node Machine with Ansible installed Managed Nodes Target servers Inventory List of servers Playbooks YAML automation files Tasks Individual actions Modules Built-in functionalities Roles Reusable scripts



How Ansible Works

- Control node connects to managed nodes via SSH
- Executes modules through tasks
- Tasks grouped into playbooks

- Inventory defines servers



Benefits

- Free & open source
- Easy to use
- No agents required
- Scalable
- Strong community support



Part A: Installation

Install via pip

pip `install` ansible

```
C:\Users\ASUS>pip install ansible
Collecting ansible
  Downloading ansible-13.5.0-py3-none-any.whl.metadata (8.1 kB)
Collecting ansible-core~=2.20.4 (from ansible)
  Downloading ansible_core-2.20.4-py3-none-any.whl.metadata (7.7 kB)
Collecting jinja2>=3.1.0 (from ansible-core~=2.20.4->ansible)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting PyYAML>=5.1 (from ansible-core~=2.20.4->ansible)
  Downloading pyyaml-6.0.3-cp312-cp312-win_amd64.whl.metadata (2.4 kB)
Collecting cryptography (from ansible-core~=2.20.4->ansible)
  Downloading cryptography-46.0.7-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Requirement already satisfied: packaging in c:\users\asus\appdata\local\programs\python\python312\lib\site-packages (from ans
,1)
Collecting resolvelib<2.0.0,>=0.8.0 (from ansible-core~=2.20.4->ansible)
  Downloading resolvelib-1.2.1-py3-none-any.whl.metadata (3.7 kB)
Collecting MarkupSafe>=2.0 (from jinja2>=3.1.0->ansible-core~=2.20.4->ansible)
  Downloading markupsafe-3.0.3-cp312-cp312-win_amd64.whl.metadata (2.8 kB)
Collecting cffi>=2.0.0 (from cryptography->ansible-core~=2.20.4->ansible)
  Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography->ansible-core~=2.20.4->ansible)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading ansible-13.5.0-py3-none-any.whl (56.1 MB)
  56.1/56.1 MB 493.0 kB/s eta 0:00:00
Downloading ansible_core-2.20.4-py3-none-any.whl (2.4 MB)
  2.4/2.4 MB 604.8 kB/s eta 0:00:00
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading pyyaml-6.0.3-cp312-cp312-win_amd64.whl (154 kB)
Downloading resolvelib-1.2.1-py3-none-any.whl (18 kB)
Downloading cryptography-46.0.7-cp311-abi3-win_amd64.whl (3.5 MB)
  3.5/3.5 MB 773.3 kB/s eta 0:00:00
Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl (183 kB)
Downloading markupsafe-3.0.3-cp312-cp312-win_amd64.whl (15 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: resolvelib, PyYAML, pycparser, MarkupSafe, jinja2, cffi, cryptography, ansible-core, ansible
[notice] A new release of pip is available: 25.0.1 -> 26.0.1
```

```
ansible --version
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/aarohi26/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/aarohi26/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

Install via apt

```
sudo apt update -y
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ sudo apt update -y
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
137 packages can be upgraded. Run 'apt list --upgradable' to see them.
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

```
sudo apt install ansible -y
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ansible is already the newest version (9.2.0+dfsg-0ubuntu5).
The following packages were automatically installed and are no longer required:
  libcgi-fast-perl libcgi-pm-perl libclone-perl libencode-locale-perl libevent-pthreads-2.1-7t64 libfcgi-bin libfcgi-perl libfcgi0t64 libhtml-parser-perl
  libhtml-tagset-perl libhtml-template-perl libhttp-date-perl libhttp-message-perl libio-html-perl libllvm19 liblwp-mediatypes-perl libmecab2 libprotobuf-lite
  libtimedate-perl liburi-perl mecab-ipadic mecab-ipadic-utf8 mecab-utils
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 115 not upgraded.
```

Test Installation

```
ansible localhost -m ping
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Expected Output: "ping": "pong"



Docker + SSH Setup

Generate SSH Key

```
ssh-keygen -t rsa -b 4096
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/aarohi26/.ssh/id_rsa): file key
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in file key
Your public key has been saved in file key.pub
The key fingerprint is:
SHA256:fUQeoT1rk0BFFzpBYxkfajRR9F+Uz2sEIIQeaZ0D6uM aarohi26@LAPTOP-4MMN5FCL
The key's randomart image is:
+---[RSA 4096]---+
|      .oo=^B*.o |
|      . = .B.@ = |
|      .o . + X o.+ |
|      . . . + = . = |
|      o S . * . o |
|      . .   o . o |
|      E           |
+---[SHA256]-----+
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```



```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ cp "file key.pub"
cp: 'file key.pub' and './file key.pub' are the same file
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ cp "file key" .
cp: 'file key' and './file key' are the same file
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Create Dockerfile

```
FROM ubuntu
```

```
RUN apt update -y
```

```
RUN apt install -y python3 python3-pip openssh-server
```

```
RUN mkdir -p /var/run/sshd
```

```
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config &
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config &&
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config
```

```
RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh
```

```
COPY id_rsa /root/.ssh/id_rsa
```

```
COPY id_rsa.pub /root/.ssh/authorized_keys
```

```
RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/authorized_keys
```

```
RUN sed -i 's@session\\s*required\\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc
```

```
EXPOSE 22
```

```
CMD ["/usr/sbin/sshd", "-D"]
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ nano Dockerfile
```

```
GNU nano 7.2 Dockerfile
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys (note: this is not secure for production!)
COPY file_key.pub /root/.ssh/authorized_keys

# Set proper permissions for keys
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# Fix for SSH login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

# Expose SSH port
EXPOSE 22

# Start SSH service when container starts
CMD ["/usr/sbin/sshd", "-D"]
```

Build Image

```
docker build -t ubuntu-server .
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ docker build -t ubuntu-server .
[+] Building 29.5s (14/14) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 1.03kB                                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest                 0.1s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                    0.0s
=> [1/9] FROM docker.io/library/ubuntu:latest@sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072fee0c 0.0s
=> resolve docker.io/library/ubuntu:latest@sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072fee0c 0.0s
=> [internal] load build context                                                0.0s
=> => transferring context: 348B                                                  0.0s
=> CACHED [2/9] RUN apt update -y                                              0.0s
=> CACHED [3/9] RUN apt install -y python3 python3-pip openssh-server          0.0s
=> CACHED [4/9] RUN mkdir -p /var/run/sshd                                     0.0s
=> CACHED [5/9] RUN mkdir -p /run/sshd && echo 'root:password' | chpasswd && sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/s 0.0s
=> CACHED [6/9] RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh                0.0s
=> CACHED [7/9] COPY file_key.pub /root/.ssh/authorized_keys                  0.0s
=> [8/9] RUN chmod 644 /root/.ssh/authorized_keys                             0.2s
=> [9/9] RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd 0.3s
=> exporting image                                                            28.8s
=> => exporting layers                                                            22.9s
=> => exporting manifest sha256:5d20459d4032af3746e653695b538bd9b98c407e2b08a329943993d326f9279a 0.0s
=> => exporting config sha256:8ed61ad7c335d196a80ff5f2e2b38237970e273d9b8ab72296cbfc423fa0e6830 0.0s
=> => exporting attestation manifest sha256:4443ea098df5586f48abb9eabad37d96a0b644b0a98b278a4f57c3e5c27ad031 0.0s
=> => exporting manifest list sha256:7e7c249bd2ed06d30d59a949f95c9840257ca506eb9272066a458026306c8346 0.0s
=> => naming to docker.io/library/ubuntu-server:latest                        0.0s
=> => unpacking to docker.io/library/ubuntu-server:latest                    5.8s
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ |
```

Run Container

```
docker run -d -p 2222:22 --name ssh-test-server ubuntu-server
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
4ff2b7f657365018adb3842a9907cb68cd3eaf76623e118e9329ef38517a67
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$
```

Get Container IP

```
docker inspect -f '{{.NetworkSettings.Networks.{{.NetworkSettings.Networks}}.IPAddress}}' ssh-test-server
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server
172.17.0.2
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$
```

Test SSH

Password:

```
ssh root@localhost -p 2222
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ ssh -i ~/.ssh/file_key root@localhost -p 2222
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:9TX0RrU1+4VtFQ5SHLLNUobmTmI6grc00pYwCCAxZR0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@4ff2b7f65736:~#
```

Key-based:

```
ssh -i ~/.ssh/id_rsa root@localhost -p 2222
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ ssh -i ~/.ssh/file_key root@localhost -p 2222
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Apr 11 04:14:21 2026 from 172.17.0.1
root@4ff2b7f65736:~#
```

Ansible with Docker

Run Multiple Servers

```
for i in {1..4}; do
    docker run -d -p 220${i}:22 --name server${i} ubuntu-server
done
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ for i in {1..4}; do
    echo -e "\n Creating server${i}\n"
    docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
    echo -e "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done
0

    Creating server1

442a1acf8cf89788d2fa5c4e0162ed61d59c30b4ffe9ad6329b55c7e144a5e7e
IP of server1 is 172.17.0.3

    Creating server2

fd3ee61ae0f91446786695d73cd5d41fcdfb349cd3420c2ffbf5985ce8884239
IP of server2 is 172.17.0.4

    Creating server3

2a01e08afd7f32274f59a3a921746c88eb7214fc0e2264e6886a15c7b2e94c55
IP of server3 is 172.17.0.5

    Creating server4

6935cd7704c8d9ac24f7779b86a0cb67117434e9eb3b8ac5d7102437234655d4
IP of server4 is 172.17.0.6
0: command not found
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$
```

Create Inventory

```
echo "[servers]" > inventory.ini
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ cat << EOF >> inventory.ini
>
>
[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

```
for i in {1..4}; do
    docker inspect -f '{{ server${i} }}' >> inventory.ini
done
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ for i in {1..4}; do
    docker stop server${i}
done
server1
server2
server3
server4
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ |
```

```
cat << EOF >> inventory.ini
```

```
[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ cat inventory.ini
[servers]
172.17.0.3
172.17.0.4
172.17.0.5
172.17.0.6
[servers]
server1 ansible_host=127.0.0.1 ansible_port=2201
server2 ansible_host=127.0.0.1 ansible_port=2202

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ |
```

Test Connectivity

```
ansible all -i inventory.ini -m ping
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ cat << EOF >> inventory.ini
>
>
[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

Create Playbook (playbook1.yml)

```
---
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
```

```
    update_cache: yes
    upgrade: dist

- name: Install packages
  apt:
    name: ["vim", "htop", "wget"]
    state: present

- name: Create test file
  copy:
    dest: /root/ansible_test.txt
    content: "Configured by Ansible on "
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ cat inventory.ini
[servers]
172.17.0.3
172.17.0.4
172.17.0.5
172.17.0.6
[servers]
server1 ansible_host=127.0.0.1 ansible_port=2201
server2 ansible_host=127.0.0.1 ansible_port=2202

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ |
```

Run Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ ansible all -i inventory.ini -m ping
server4 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
server3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
server1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
server2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ |
```

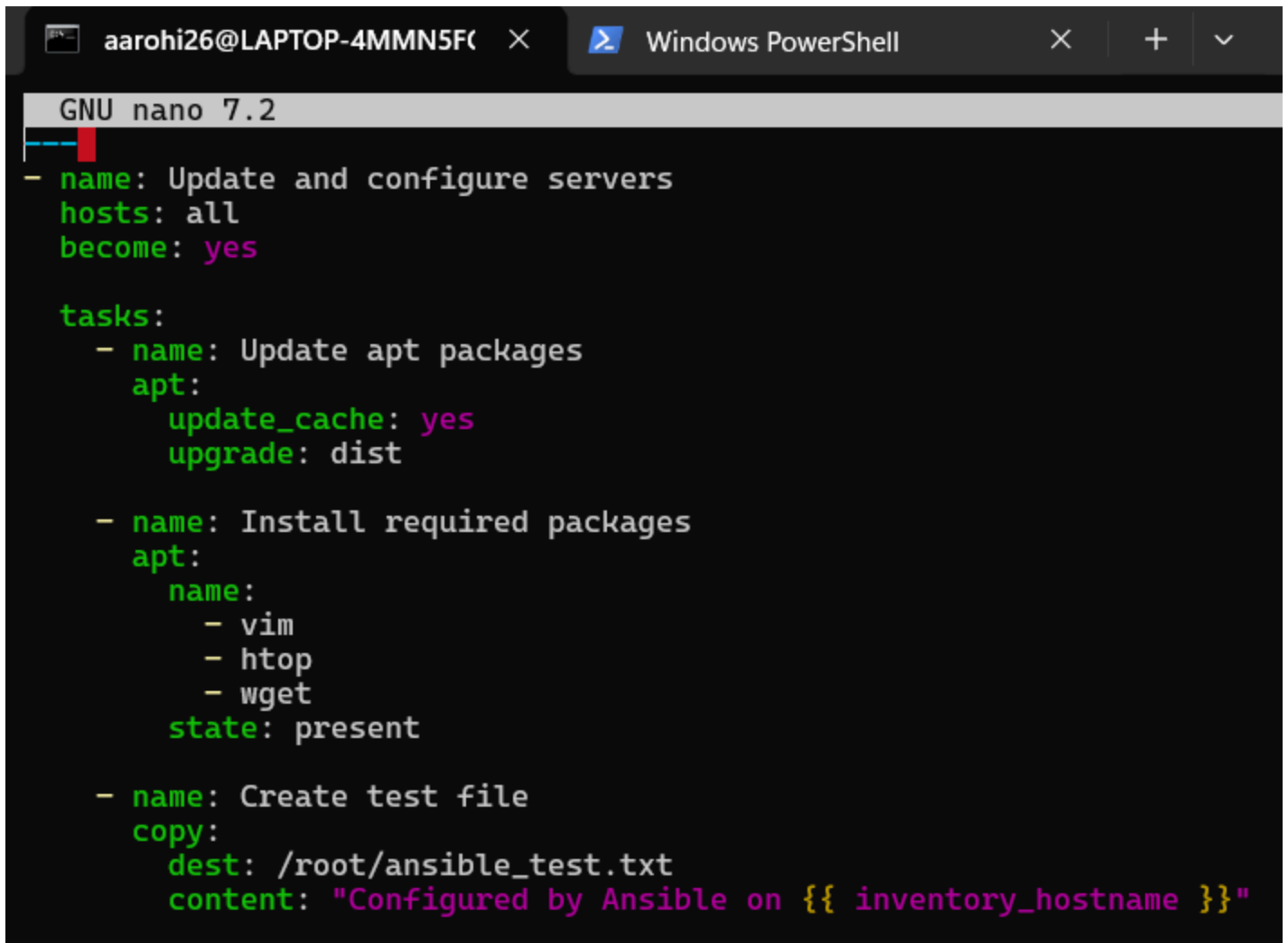
Verify Output

```
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/docker-ssh$ nano update.yml
```

Cleanup

```
for i in {1..4}; do docker rm -f server${i}; done
```

The screenshot shows a Windows PowerShell terminal window with a tab labeled 'aaroji26@LAPTOP-4MMN5F...' and 'Windows PowerShell'. Inside the terminal, the GNU nano 7.2 editor is open, displaying an Ansible playbook. The playbook has a 'name' of 'Update and configure servers', 'hosts' set to 'all', and 'become' set to 'yes'. It contains three tasks: 1) 'Update apt packages' using the 'apt' module with 'update_cache: yes' and 'upgrade: dist'. 2) 'Install required packages' using the 'apt' module with a list of packages (vim, htop, wget) and 'state: present'. 3) 'Create test file' using the 'copy' module with 'dest: /root/ansible_test.txt' and 'content: "Configured by Ansible on {{ inventory_hostname }}"'.

```
GNU nano 7.2
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name:
          - vim
          - htop
          - wget
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on {{ inventory_hostname }}"
```



Need for Ansible

- Scalability
- Consistency
- Efficiency
- Idempotency
- Infrastructure as Code



Key Features

- Agentless
- Idempotent
- YAML-based

- Large module library
- Push-based



Conclusion

Ansible simplifies server management by automating repetitive tasks, ensuring consistency, and enabling scalable infrastructure management.

Containerization_and_DevOps

SonarQube Code Quality Analysis using Docker, Maven, and Jenkins

Objective

To set up SonarQube using Docker, analyze a Java application for bugs, vulnerabilities, and code smells, and integrate it with Jenkins for CI/CD-based quality checks.

Tools & Technologies Used

- Docker & Docker Compose
- SonarQube (LTS Community Edition)
- PostgreSQL
- Maven
- Jenkins
- Java (Sample Application)

Step 1: Setup SonarQube Environment

Create Docker Network

```
docker network create sonarqube-lab
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker network create sonarqube-lab  
fa7961ac9251363819222ba109ab6c3ed5972f0f200344c8c5e7eb299ad59933
```

Start PostgreSQL Database

```
docker run -d \  
  --name sonar-db \  
  sonarqube/sonarqube-lts-community-edition
```

```
--network sonarqube-lab \
-e POSTGRES_USER=sonar \
-e POSTGRES_PASSWORD=sonar \
-e POSTGRES_DB=sonarqube \
-v sonar-db-data:/var/lib/postgresql/data \
postgres:13
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name sonar-db \
--network sonarqube-lab \
-e POSTGRES_USER=sonar \
-e POSTGRES_PASSWORD=sonar \
-e POSTGRES_DB=sonarqube \
-v sonar-db-data:/var/lib/postgresql/data \
postgres:13
Unable to find image 'postgres:13' locally
13: Pulling from library/postgres
d7ecded7702a: Pull complete
bad194149a82: Pull complete
fcfd49225910: Pull complete
ba168ff08d42: Pull complete
5f1cded65589: Pull complete
947bdcac0ea3: Pull complete
4d2c7a7b3dba: Pull complete
097f34b018c0: Pull complete
8cb000c698bc: Pull complete
23a39b1003e9: Pull complete
7d0650bb210f: Pull complete
21f7a31efcb7: Pull complete
ae52c8ce1a35: Pull complete
80e8d95fdf53: Pull complete
c7bebc4b5ec1: Download complete
d932a4a7df6c: Download complete
Digest: sha256:4689940c683801b4ab839ab3b0a0a3555a5fe425371422310944e89eca7d8068
Status: Downloaded newer image for postgres:13
0cd5dc7ba9767be7c5185a2aff9ce7efc7c0cd4a1880de50338cf3aaf6f1c11a
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Start SonarQube Server

```
docker run -d \
--name sonarqube \
--network sonarqube-lab \
-p 9000:9000 \
-e SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonarqube \
-e SONAR_JDBC_USERNAME=sonar \
-e SONAR_JDBC_PASSWORD=sonar \
-v sonar-data:/opt/sonarqube/data \
-v sonar-extensions:/opt/sonarqube/extensions \
sonarqube:lts-community
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name sonarqube \
--network sonarqube-lab \
-p 9000:9000 \
-e SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonarqube \
-e SONAR_JDBC_USERNAME=sonar \
-e SONAR_JDBC_PASSWORD=sonar \
-v sonar-data:/opt/sonarqube/data \
-v sonar-extensions:/opt/sonarqube/extensions \
sonarqube:lts-community
Unable to find image 'sonarqube:lts-community' locally
lts-community: Pulling from library/sonarqube
4f4fb700ef54: Pull complete
1df735f481ad: Pull complete
c7ad1fe61e07: Pull complete
eb27e3a98da1: Pull complete
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
5d5a1fad7028: Pull complete
60d98d907669: Pull complete
ce2bd68be4fc: Download complete
600f318704dc: Download complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
0d40c946078334f6b91ae821044dc13b9686c8cfc3691dbfb03e0f6c51a1b568
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

Verify Logs

docker logs -f sonarqube

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker logs -f sonarqube
2026.04.18 03:13:20 INFO app[[[o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
2026.04.18 03:13:20 INFO app[[[o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:45103]
2026.04.18 03:13:20 INFO app[[[o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/sonarqube/elasticsearch/b
in/elasticsearch
2026.04.18 03:13:20 INFO app[[[o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
2026.04.18 03:13:22 INFO es[[[o.e.n.Node] version[7.17.15], pid[29], build[default/tar/0b8ecfb4378335f4689c4223d1f115f16bef3ba/2023-11-10T22:03:46.9873990
16Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8]
2026.04.18 03:13:22 INFO es[[[o.e.n.Node] JVM home [/opt/java/openjdk]
2026.04.18 03:13:22 INFO es[[[o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs/es_hs_err_pi
d%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -Dfile.encoding
=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimization=true,
-Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shutdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.f
ormatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -Des.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:MaxDirectMemory
Size=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavo
r=default, -Des.distribution.type=tar, -Des.bundled_jdk=false]
2026.04.18 03:13:22 INFO es[[[o.e.p.PluginsService] loaded module [analysis-common]
2026.04.18 03:13:22 INFO es[[[o.e.p.PluginsService] loaded module [lang-painless]
2026.04.18 03:13:22 INFO es[[[o.e.p.PluginsService] loaded module [parent-join]
2026.04.18 03:13:22 INFO es[[[o.e.p.PluginsService] loaded module [reindex]
2026.04.18 03:13:22 INFO es[[[o.e.p.PluginsService] loaded module [transport-netty4]
2026.04.18 03:13:22 INFO es[[[o.e.p.PluginsService] no plugins loaded
2026.04.18 03:13:22 INFO es[[[o.e.e.NodeEnvironment] using [1] data paths, mounts [[/opt/sonarqube/data (/dev/sdf)], net usable_space [939.8gb], net total
_space [1006.8gb], types [ext4]
2026.04.18 03:13:22 INFO es[[[o.e.e.NodeEnvironment] heap size [512mb], compressed ordinary object pointers [true]
2026.04.18 03:13:22 INFO es[[[o.e.n.Node] node name [sonarqube], node ID [piDVpyRsQxGKaDl7Kjmiuw], cluster name [sonarqube], roles [data_frozen, master, re
mote_cluster_client, data, data_content, data_hot, data_warm, data_cold, ingest]
2026.04.18 03:13:22 WARN es[[[o.e.t.ThreadPool] timer thread slept for [26.3s/26399ms] on absolute clock which is above the warn threshold of [5000ms]
2026.04.18 03:13:22 WARN es[[[o.e.t.ThreadPool] absolute clock went backwards by [25.9s/25996ms] while timer thread was sleeping
2026.04.18 03:13:25 INFO es[[[o.e.t.NettyAllocator] creating NettyAllocator with the following configs: [name=unpooled, suggested_max_allocation_size=256kb
, factors={es.unsafe.use_unpooled_allocator=null, g1gc_enabled=true, g1gc_region_size=1mb, heap_size=512mb}]
2026.04.18 03:13:25 INFO es[[[o.e.i.r.RecoverySettings] using rate limit [40mb] with [default=40mb, read=0b, write=0b, max=0b]
2026.04.18 03:13:25 INFO es[[[o.e.d.DiscoveryModule] using discovery type [zen] and seed hosts providers [settings]
2026.04.18 03:13:25 INFO es[[[o.e.g.DanglingIndicesState] gateway.auto_import_dangling_indices is disabled, dangling indices will not be automatically dete
cted or imported and must be managed manually
2026.04.18 03:13:25 INFO es[[[o.e.n.Node] initialized
2026.04.18 03:13:25 INFO es[[[o.e.n.Node] starting ...
2026.04.18 03:13:25 INFO es[[[o.e.t.TransportService] publish_address [127.0.0.1:45103], bound_addresses [127.0.0.1:45103]
2026.04.18 03:13:25 INFO es[[[o.e.b.BootstrapChecks] explicitly enforcing bootstrap checks
2026.04.18 03:13:25 INFO es[[[o.e.c.c.Coordinator] setting initial configuration to VotingConfiguration{piDVpyRsQxGKaDl7Kjmiuw}]
```

Access SonarQube

- URL: `http://localhost:9000`
- Default Login: `admin / admin`

The first screenshot shows the SonarQube maintenance page at `localhost:9000/maintenance?return_to=%2F`. It displays a message: "SonarQube is up" and "All systems operational." with a [Home](#) link.

The second screenshot shows the SonarQube login page at `localhost:9000/sessions/new?return_to=%2F`. It has a title "Log in to SonarQube" and fields for "Login" and "Password". There are "Log in" and "Cancel" buttons.

The third screenshot shows the SonarQube project creation page at `localhost:9000/projects/create`. It has a title "How do you want to create your project?" and a sub-header "Do you want to benefit from all of SonarQube's features (like repository import and Pull Request decoration)? Create your project from your favorite DevOps platform. First, you need to set up a DevOps platform configuration." Below this are five options: "From Azure DevOps", "From Bitbucket Server", "From Bitbucket Cloud", "From GitHub", and "From GitLab". Each option has a "Set up global configuration" link. At the bottom, there is a "Manually" option with a code icon and the text "Are you just testing or have an advanced use-case? Create a project manually."



Alternative: Docker Compose Setup

```
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    restart: unless-stopped
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

networks:
  sonarqube-lab:
    driver: bridge
```

Run:

```
docker compose up -d
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ docker compose up -d
WARN[0000] /mnt/c/Users/ASUS/sonarqube-lab/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential
confusion
[+] up 2/2
✔Container sonar-db Created 0.1s
✔Container sonarqube Created 0.2s
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ |
```



Step 2: Create Sample Java Application

Create Project Structure

```
mkdir -p sample-java-app/src/main/java/com/example
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ mkdir -p sample-java-app/src/main/java/com/example
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ cat > sample-java-app/src/main/java/com/example/Calculator.java << 'EOF'
```

Add Java Code (with issues)

(Create Calculator.java as provided in experiment)


```

aaroji26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ cat > sample-java-app/src/main/java/com/example/Calculator.java << 'EOF'
> package com.example;

import java.util.ArrayList;
import java.util.List;

public class Calculator {

    // Bug: Division by zero not handled
    public int divide(int a, int b) {
        return a / b; // Bug: Potential division by zero
    }

    // Code Smell: Unused variable
    public int add(int a, int b) {
        int result = a + b;
        int unused = 100; // Code smell: Unused variable
        return result;
    }

    // Vulnerability: SQL injection risk
    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId; // Vulnerability: SQL injection
        return query;
    }

    // Code Smell: Duplicate code
    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    // Duplicate code (same as multiply method)
    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }
} // Empty catch block - swallowing exception if name is null
aaroji26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ cat sample-java-app/src/main/java/com/example/Calculator.java
package com.example;

import java.util.ArrayList;
import java.util.List;

public class Calculator {

```

```

j
aaruhi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab$ cat > sample-java-app/pom.xml << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.organization>my-org</sonar.organization>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>your-sonar-token</sonar.login>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>
EOF

```

Step 3: Maven Configuration

Create pom.xml

(Use provided pom.xml)

Build Project

```

cd sample-java-app
mvn clean install

```

```

aaroHi26@LAPTOP-4MMNSFCL:/mnt/c/Users/ASUS/sonarqube-lab$ cd sample-java-app
mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] < com.example:sample-app >
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] [ jar ]
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.pom (10 kB at 14 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/24/maven-plugins-24.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/24/maven-plugins-24.pom (11 kB at 140 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.jar (43 kB at 275 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.pom (6.4 kB at 60 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.jar (27 kB at 270 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.pom (27 kB at 311 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.pom (766 B at 29 B/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-parent/1.3/hamcrest-parent-1.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-parent/1.3/hamcrest-parent-1.3.pom (2.0 kB at 18 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar (385 kB)
Downloading from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar (45 kB)
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ sample-app ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ sample-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /mnt/c/Users/ASUS/sonarqube-lab/sample-java-app/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ sample-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.5.1/plexus-utils-3.5.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.5.1/plexus-utils-3.5.1.pom (640 kB)
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /mnt/c/Users/ASUS/sonarqube-lab/sample-java-app/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ sample-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /mnt/c/Users/ASUS/sonarqube-lab/sample-java-app/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ sample-app ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ sample-app ---
[INFO] No tests to run.
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ sample-app ---
[INFO] Building jar: /mnt/c/Users/ASUS/sonarqube-lab/sample-java-app/target/sample-app-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ sample-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.pom (2.5 kB at 43 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/3.1/plexus-3.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/3.1/plexus-3.1.pom (19 kB at 345 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.pom (1.1 kB at 22 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-components/1.1.7/plexus-components-1.1.7.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-components/1.1.7/plexus-components-1.1.7.pom (5.0 kB at 77 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/1.0.8/plexus-1.0.8.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/1.0.8/plexus-1.0.8.pom (7.2 kB at 99 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-container-default/1.0-alpha-8/plexus-container-default-1.0-alpha-8.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-container-default/1.0-alpha-8/plexus-container-default-1.0-alpha-8.pom (7.3 kB at 119 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.jar (12 kB at 198 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.jar (230 kB at 3.5 MB/s)
[INFO] Installing /mnt/c/Users/ASUS/sonarqube-lab/sample-java-app/target/sample-app-1.0-SNAPSHOT.jar to /home/aaroHi26/.m2/repository/com/example/sample-app/1.0-SNAPSHOT/sample-app-1.0-SNAPSHOT.jar
[INFO] Installing /mnt/c/Users/ASUS/sonarqube-lab/sample-java-app/pom.xml to /home/aaroHi26/.m2/repository/com/example/sample-app/1.0-SNAPSHOT/sample-app-1.0-SNAPSHOT.pom
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 10.167 s
[INFO] Finished at: 2026-04-18T03:41:50Z
[INFO]
aaroHi26@LAPTOP-4MMNSFCL:/mnt/c/Users/ASUS/sonarqube-lab/sample-java-app$

```

Step 4: SonarQube Scanner Setup

Run Scanner via Docker

```

docker run -d \
  --name sonar-scanner \
  --network sonarqube-lab \
  -v $(pwd)/sample-java-app:/usr/src \
  sonarsource/sonar-scanner-cli \
  sleep infinity

```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab/sample-java-app$ docker run -d \
--name sonar-scanner \
--network sonarqube-lab \
-v $(pwd):/usr/src \
sonarsource/sonar-scanner-cli \
sleep infinity
520fa5f51b73cc2a40d42e7fcbcd7c292643ebc3b7fe26545d1474085b5565b
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/sonarqube-lab/sample-java-app$ |
```



Step 5: Generate Token

- Go to SonarQube Dashboard
- My Account → Security → Generate Token
- Copy token

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name: Aarohi-token Type: User Token Expires in: No expiration [Generate](#)

Name	Type	Project	Last use	Created	Expiration
No tokens					

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name: Enter Token Name Type: Select Token Type Expires in: 30 days [Generate](#)

Message: New token "token_aarohi" has been created. Make sure you copy it now, you won't be able to see it again!

[Copy](#) `squ_88fe0fc419310bd6498583125c2e690c1dbd9484`

Name	Type	Project	Last use	Created	Expiration
token_aarohi	User		Never	April 24, 2026	- Revoke



Step 6: Run Code Analysis

```
docker exec -e SONAR_TOKEN="your-token" \  
  sonar-scanner \  
  sonar-scanner \  
  -Dsonar.host.url=http://sonarqube:9000 \  
  -Dsonar.projectKey=sample-java-app \  
  -Dsonar.sources=/usr/src/src
```

```
aaron126@LAPTOP-4HNN5FLL:/mnt/c/Users/ASUS$ docker exec -e SONAR_TOKEN="squ_6d9bee3850071507b0e51e0ca59a3c4c02ed01/1" sonar-scanner sonar-scanner -Dsonar.host.url=http://sonarqube:9000 -Dsonar.projectKey=sample-java-app -Dsonar.sources=/usr/src/src  
08 -Dsonar.projectKey=sample-java-app -Dsonar.sources=/usr/src/src  
05:53:10.096 INFO Scanner configuration file: /opt/sonar-scanner/conf/sonar-scanner.properties  
05:53:10.101 INFO Project root configuration file: NONE  
05:53:10.119 INFO SonarScanner CLI 8.0.1.6346  
05:53:10.120 INFO Linux 6.6.87.2-microsoft-standard-WSL2 amd64  
05:53:15.770 ERROR Failed to query server version: Call to URL [http://sonarqube:9000/api/v2/analysis/version] failed: null  
05:53:15.771 INFO EXECUTION FAILURE  
05:53:15.772 INFO Total time: 5.681s
```



Step 7: View Results

Open:

<http://localhost:9000/dashboard?id=sample-java-app>

```

06:18:22.470 ERROR
06:18:22.470 ERROR Re-run SonarScanner CLI using the -X switch to enable full debug logging.
aaron@1248:~$ docker exec sonar-scanner sh -c 'sonar-scanner \
-Dsonar.host.url=http://host.docker.internal:9000 \
-Dsonar.projectKey=sample-java-app \
-Dsonar.sources=/usr/src/src \
-Dsonar.logLevel=DEBUG -Dsonar.ci=11dbd498583125c2a690c1dbd9089'
06:13:22.970 INFO Scanner configuration file: /opt/sonar-scanner/conf/sonar-scanner.properties
06:13:22.587 INFO Project root configuration file: NONE
06:13:22.603 INFO SonarScanner CLI 8.0.1.6346
06:13:22.610 INFO Linux 6.6.87.2-microsoft-standard-WSL2 amd64
06:13:24.004 INFO Communicating with SonarQube Server 9.9.8.100196
06:13:24.527 INFO Load global settings
06:13:24.605 INFO Load global settings (done) | time=79ms
06:13:24.607 INFO Server id: 74C15348-A22a1JVU4c1eMAZtvtG1U
06:13:24.614 INFO User cache: /opt/sonar-scanner/.sonar/cache
06:13:24.619 INFO Load/download plugins
06:13:24.619 INFO Load plugins index
06:13:24.681 INFO Load plugins index (done) | time=62ms
06:13:31.614 INFO Load/download plugins (done) | time=6995ms
06:13:32.124 INFO Process project properties
06:13:32.139 INFO Process project properties (done) | time=15ms
06:13:32.140 INFO Execute project builders
06:13:32.145 INFO Execute project builders (done) | time=2ms
06:13:32.161 INFO Project key: sample-java-app
06:13:32.161 INFO Base dir: /usr/src
06:13:32.161 INFO Working dir: /usr/src/.scannerwork
06:13:32.186 INFO Load project settings for component key: 'sample-java-app'
06:13:32.350 WARN SCM provider autodetection failed. Please use "sonar.scm.provider" to define SCM of your project, or disable the SCM Sensor in the project settings.
06:13:32.368 INFO Load quality profiles
06:13:32.458 INFO Load quality profiles (done) | time=90ms
06:13:32.473 INFO Load active rules
06:13:34.734 INFO Load active rules (done) | time=2261ms
06:13:34.738 INFO Load analysis cache
06:13:34.748 INFO Load analysis cache (404) | time=10ms
06:13:34.840 INFO Load project repositories
06:13:34.860 INFO Load project repositories (done) | time=20ms
06:13:34.880 INFO Indexing files
06:13:34.889 INFO Project configuration:
06:14:04.194 INFO 1 file indexed
06:14:04.195 INFO Quality profile for java: Sonar way
06:14:04.195 INFO ----- Run sensors on module sample-java-app
06:14:04.270 INFO Load metrics repository
06:14:04.313 INFO Load metrics repository (done) | time=40ms
06:13:36.880 INFO Sensor JavaSensor [java]
06:13:36.885 INFO Configured Java source version (sonar.java.source): none
06:13:36.888 INFO JavaClasspath initialization
06:13:36.889 INFO JavaClasspath initialization (done) | time=1ms
06:13:36.889 INFO JavaTestClasspath initialization
06:13:36.890 INFO JavaTestClasspath initialization (done) | time=1ms
06:13:36.896 INFO Server-side caching is enabled. The Java analyzer will not try to leverage data from a previous analysis.
06:13:36.898 INFO Using ECJ batch to parse 1 Main java source files with batch size 101 MB.
06:13:37.167 INFO Starting batch processing.
06:14:07.270 INFO The Java analyzer cannot skip unchanged files in this context. A full analysis is performed for all files.
06:13:38.906 INFO 100% analyzed
06:13:38.906 INFO Batch processing: Done.
06:13:38.908 INFO Did not optimize analysis for any files, performed a full analysis for all 1 files.
06:13:38.910 INFO No "Test" source files to scan.
06:13:38.910 INFO No "Generated" source files to scan.
06:13:38.910 INFO Sensor JavaSensor [java] (done) | time=2030ms
06:13:38.910 INFO Sensor JaCoCo XML Report Importer [jacoco]
06:13:38.914 INFO 'sonar.coverage.jacoco.xmlReportPaths' is not defined. Using default locations: target/site/jacoco/jacoco.xml,target/site/jacoco-it/jacoco.xml,build/reports/jacoco/test/jacocoTestReport.xml
06:13:38.920 INFO No report imported, no coverage information will be imported by JaCoCo XML Report Importer
06:13:38.920 INFO Sensor JaCoCo XML Report Importer [jacoco] (done) | time=10ms
06:13:38.920 INFO Sensor CSS Rules [javascript]
06:13:38.929 INFO No CSS, PHP, HTML or VueJS files are found in the project. CSS analysis is skipped.
06:13:38.929 INFO Sensor CSS Rules [javascript] (done) | time=1ms
06:13:38.929 INFO Sensor C# Project Type Information [csharp]
06:13:38.931 INFO Sensor C# Project Type Information [csharp] (done) | time=2ms
06:13:38.931 INFO Sensor C# Analysis Log [csharp]
06:13:38.991 INFO Sensor C# Analysis Log [csharp] (done) | time=60ms
06:13:38.991 INFO Sensor C# Properties [csharp]
06:13:38.991 INFO Sensor C# Properties [csharp] (done) | time=0ms
06:13:38.991 INFO Sensor SurefireSensor [java]
06:13:38.992 INFO parsing [/usr/src/target/surefire-reports]
06:13:38.995 INFO Sensor SurefireSensor [java] (done) | time=4ms
06:13:38.996 INFO Sensor HTML [web]
06:13:38.982 INFO Sensor HTML [web] (done) | time=6ms
06:13:39.003 INFO Sensor TextAndSecretsSensor [text]
06:13:39.036 INFO 1 source file to be analyzed
06:13:39.108 INFO 1/1 source file has been analyzed
06:13:39.108 INFO Sensor TextAndSecretsSensor [text] (done) | time=105ms
06:13:39.109 INFO Sensor VB.NET Project Type Information [vbnet]
06:13:39.109 INFO Sensor VB.NET Project Type Information [vbnet] (done) | time=1ms
06:13:39.109 INFO Sensor VB.NET Analysis Log [vbnet]
06:13:39.131 INFO Sensor VB.NET Analysis Log [vbnet] (done) | time=22ms
06:13:39.131 INFO Sensor VB.NET Properties [vbnet]
06:13:39.131 INFO Sensor VB.NET Properties [vbnet] (done) | time=0ms
06:13:39.131 INFO Sensor Iac Docker Sensor [iac]
06:13:39.134 INFO 0 source files to be analyzed
06:13:39.209 INFO 0/0 source files have been analyzed
06:13:39.210 INFO Sensor Iac Docker Sensor [iac] (done) | time=79ms
06:13:39.213 INFO ----- Run sensors on project
06:13:39.270 INFO Sensor Analysis Warnings Import [csharp]
06:13:39.280 INFO Sensor Analysis Warnings Import [csharp] (done) | time=1ms
06:13:39.280 INFO Sensor Zero Coverage Sensor

```

Observed Issues:

- Bugs: Division by zero, Null pointer
- Vulnerability: SQL Injection
- Code Smells:
 - Unused variables
 - Duplicate code
 - Too many parameters
 - Empty catch block

Metrics:

- Bugs: ~8
- Vulnerabilities: 1
- Code Smells: ~12
- Duplications: 2 blocks
- Test Coverage: 0%
- Technical Debt: ~2 hours

Challenges Faced & Solutions

1. SonarQube Not Starting

- Cause: Low system memory
- Solution:

```
sysctl -w vm.max_map_count=262144
```

2. Database Connection Failure

- Cause: Incorrect JDBC URL
- Solution: Ensure correct service name (sonar-db)

3. Scanner Not Connecting

- Cause: Network mismatch
- Solution: Use same Docker network

4. Token Authentication Error

- Cause: Invalid token
- Solution: Regenerate token



Conclusion

This experiment demonstrated how to:

- Deploy SonarQube using Docker
- Analyze code quality issues in a Java application
- Identify bugs, vulnerabilities, and code smells
- Integrate SonarQube with Jenkins for CI/CD
- Enforce Quality Gates before deployment

SonarQube helps improve code reliability, maintainability, and security by detecting issues early in the development lifecycle.



Future Improvements

- Add unit test coverage
- Fix detected issues
- Integrate with GitHub Actions
- Enable automated reports

Containerization_and_DevOps

Experiment 11: Orchestration using Docker Compose & Docker Swarm (Continuation of Experiment 6)

PART A – CONCEPT CONTINUATION (Simple Explanation)

From Experiment 6, docker run runs a single container but requires manual handling and has no coordination, while Docker Compose runs multiple containers together but only on a single machine and without auto-healing. The new concept is orchestration, which means automatic management of containers including scaling (increase/decrease containers), self-healing (restart failed containers), load balancing (distribute traffic), and multi-host deployment. The progression path is: docker run → Docker Compose → Docker Swarm → Kubernetes. This experiment focuses on moving from Compose to Swarm.

PART B – PRACTICAL (EXTENSION OF EXPERIMENT 6)

Prerequisites

- Docker installed with Swarm enabled
- docker-compose.yml file (WordPress + MySQL)

```
version: '3.9'
services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
  wordpress:
```

```

image: wordpress:latest
container_name: wordpress_app
depends_on:
  - db
ports:
  - "8080:80"
restart: always
environment:
  WORDPRESS_DB_HOST: db:3306
  WORDPRESS_DB_USER: wpuser
  WORDPRESS_DB_PASSWORD: wppass
  WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html
volumes:
  db_data:
  wp_data:

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ dir
docker-compose.yml

```

```

GNU nano 7.2 docker-compose.yml
version: '3.9'
services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass

```

[Read 33 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy

Task 1: Check Current State

```

docker compose down -v
docker ps

```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker compose down -v
docker ps
WARN[0000] /mnt/c/Users/ASUS/Experiment11/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
b07bf5fc3d11   sonarqube:lts-community             "/opt/sonarqube/dock...  13 days ago   Up About a minute   0.0.0.0
:9000->9000/tcp, [::]:9000->9000/tcp   sonarqube
eb6cb118b716   postgres:13                         "docker-entrypoint.s..." 13 days ago   Up About a minute   5432/tcp
P
24f1d004ba3a   containerassgn-mybackend            "uvicorn main:app --..." 4 weeks ago   Up About a minute   0.0.0.0
:8000->8000/tcp, [::]:8000->8000/tcp   containerassgn_mybackend
48bbb6b983c1   containerassgn-mydatabase          "docker-entrypoint.s..." 4 weeks ago   Up About a minute (healthy)
containerassgn_mydatabase
732c618607a6   projectassignment-backend           "docker-entrypoint.s..." 4 weeks ago   Up About a minute   0.0.0.0
:3000->3000/tcp, [::]:3000->3000/tcp   project_node_backend
f98234b377c0   projectassignment-database          "docker-entrypoint.s..." 4 weeks ago   Up About a minute (healthy) 5432/tcp
P
project_postgres_db
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$
```

Task 2: Initialize Docker Swarm

```
docker swarm init
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker swarm init
Swarm initialized: current node (21qw67z8v81clg7azuasadjwd) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-6bxt2v5od0abgm2atu0mlmnrq4ekzjezacg6tw63qhar9jmaw-0e0jy9i8taew88cmy3hhglrgb 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

```
docker node ls
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker node ls
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
21qw67z8v81clg7azuasadjwd *  docker-desktop  Ready     Active           Leader             29.2.0
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$
```

Task 3: Deploy as a Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
Creating service wpstack_db
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$
```

Task 4: Verify Deployment

```
docker service ls
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
mqdb8hqimqa6      wpstack_db          replicated          1/1                 mysql:5.7
o97bkdc7ysb       wpstack_wordpress    replicated          1/1                 wordpress:latest     *:8080->80/tcp
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$
```

```
docker service ps wpstack_wordpress
```

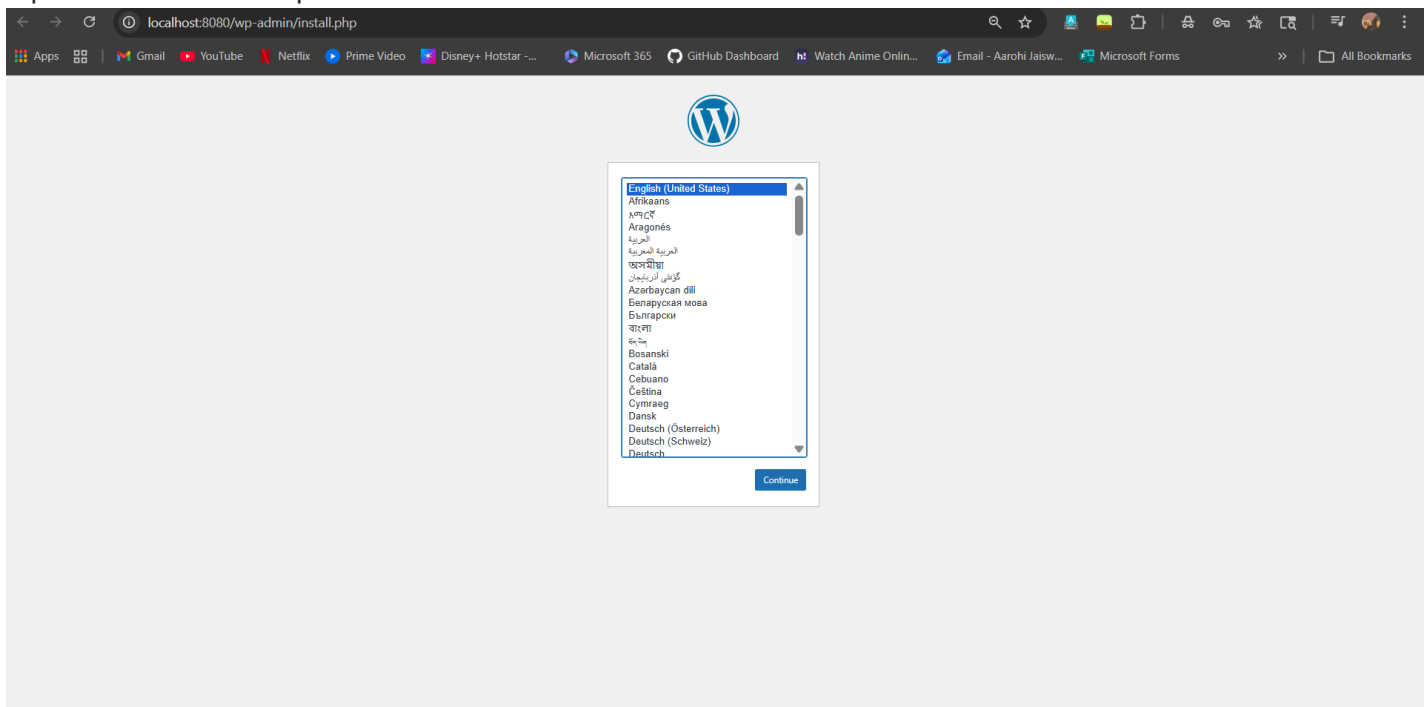
```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker service ps wpstack_wordpress
ID                NAME                IMAGE                NODE                DESIRED STATE      CURRENT STATE      ERROR
n9you5nf9g8p      wpstack_wordpress.1  wordpress:latest     docker-desktop      Running             Running 27 seconds ago
```

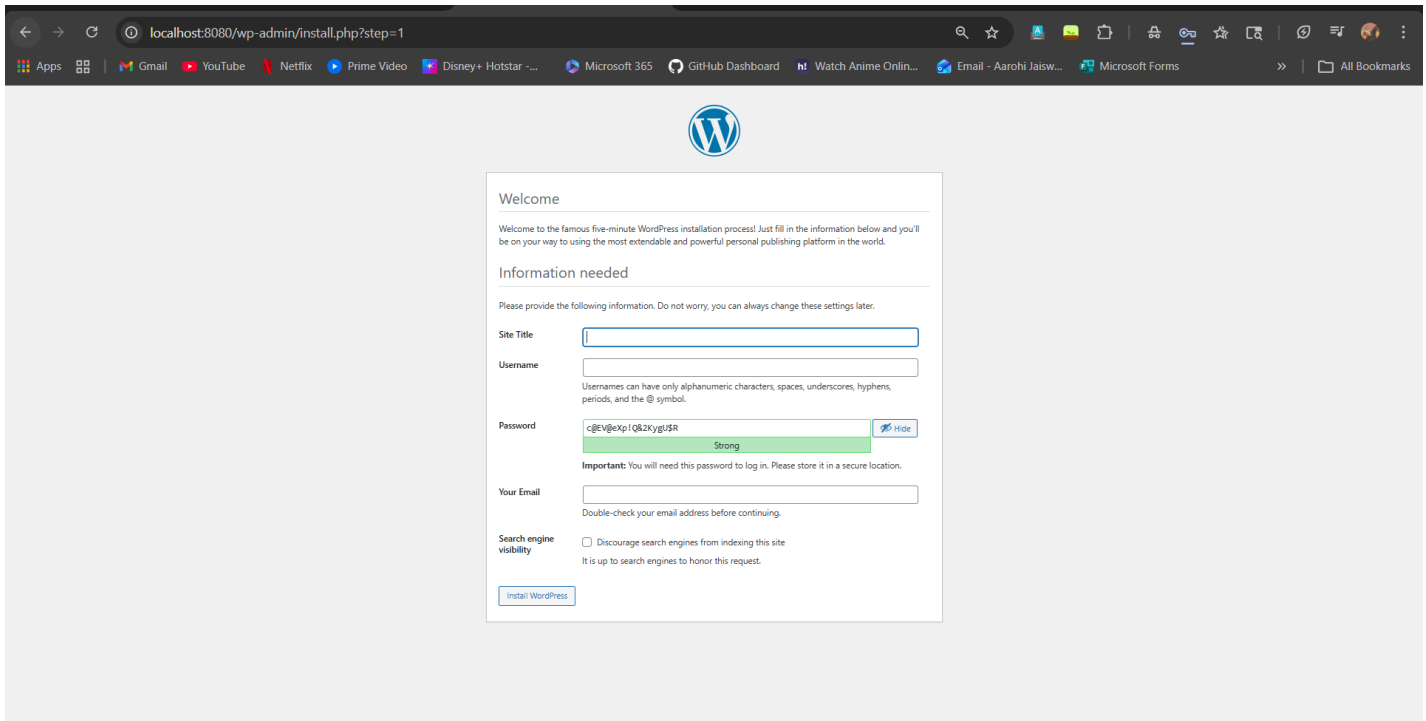
```
docker ps
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
b8f36f4fa3c6   wordpress:latest     "docker-entrypoint.s..." 2 minutes ago    Up 2 minutes    80/tcp
afb69da49caf   mysql:5.7            "docker-entrypoint.s..." 3 minutes ago    Up 3 minutes    3306/tcp
33060/tcp
b07bf5fc3d11   sonarqube:lts-commu... "/opt/sonarqube/dock..." 13 days ago      Up 9 minutes    0.0.0.0:9000->9000
```

Task 5: Access WordPress

Open in browser: <http://localhost:8080>





Task 6: Scale the Application

```
docker service scale wpstack_wordpress=3
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ |
```

```
docker service ls
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker service ls
ID                NAME                MODE                REPLICAS    IMAGE                PORTS
mqdb8hqimqa6      wpstack_db          replicated          1/1          mysql:5.7
o97bkdc7ysb       wpstack_wordpress   replicated          3/3          wordpress:latest     *:8080->80/tcp
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ |
```

```
docker service ps wpstack_wordpress
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker service ps wpstack_wordpress
ID                PORTS                NAME                IMAGE                NODE                DESIRED STATE    CURRENT STATE    ERROR
n9you5nf9g8p      wpstack_wordpress.1  wordpress:latest    docker-desktop       Running           Running 15 minutes ago
ohvplnhag33t      wpstack_wordpress.2  wordpress:latest    docker-desktop       Running           Running 5 minutes ago
epj0t57z58wb      wpstack_wordpress.3  wordpress:latest    docker-desktop       Running           Running 5 minutes ago
```

```
docker ps | grep wordpress
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker ps | grep wordpress
11569571d2c4   wordpress:latest   "docker-entrypoint.s..." 4 minutes ago   Up 4 minutes   80/tcp
6bde4cb3e0c6   wordpress:latest   "docker-entrypoint.s..." 4 minutes ago   Up 4 minutes   80/tcp
b8f36f4fa3c6   wordpress:latest   "docker-entrypoint.s..." 14 minutes ago  Up 14 minutes  80/tcp
wpstack_wordpress.3.epj0t57z58wb2hnpwe6ppx9pl
wpstack_wordpress.2.ohvplnhag33t7b50ozmb4aqpw
wpstack_wordpress.1.n9you5nf9g8pizt54dn1wgh04
```

Task 7: Test Self-Healing

```
docker ps | grep wordpress
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker ps | grep wordpress
11569571d2c4   wordpress:latest   "docker-entrypoint.s..." 4 minutes ago   Up 4 minutes   80/tcp
6bde4cb3e0c6   wordpress:latest   "docker-entrypoint.s..." 4 minutes ago   Up 4 minutes   80/tcp
b8f36f4fa3c6   wordpress:latest   "docker-entrypoint.s..." 14 minutes ago  Up 14 minutes  80/tcp
wpstack_wordpress.3.epj0t57z58wb2hnpwe6ppx9pl
wpstack_wordpress.2.ohvplnhag33t7b50ozmb4aqpw
wpstack_wordpress.1.n9you5nf9g8pizt54dn1wgh04
```

```
docker kill <container-id>
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker kill 11569571d2c4
11569571d2c4
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$
```

```
docker service ps wpstack_wordpress
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker service ps wpstack_wordpress
ID            NAME            IMAGE            NODE            DESIRED STATE   CURRENT STATE           ERROR               PORTS
n9you5nf9g8p  wpstack_wordpress.1  wordpress:latest  docker-desktop  Running         Running 22 minutes ago
ohvplnhag33t  wpstack_wordpress.2  wordpress:latest  docker-desktop  Running         Running 12 minutes ago
y89r0z4mq2jx  wpstack_wordpress.3  wordpress:latest  docker-desktop  Running         Running 17 seconds ago
epj0t57z58wb  wpstack_wordpress.3  wordpress:latest  docker-desktop  Shutdown       Failed 22 seconds ago   "task: non-zero exit (137)"
```

```
docker ps | grep wordpress
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker ps | grep wordpress
638b0c0438f6   wordpress:latest   "docker-entrypoint.s..." About a minute ago   Up About a minute   80/tcp
wpstack_wordpress.3.y89r0z4mq2jxcfr4xc04m8267
6bde4cb3e0c6   wordpress:latest   "docker-entrypoint.s..." 13 minutes ago      Up 13 minutes       80/tcp
wpstack_wordpress.2.ohvplnhag33t7b50ozmb4aqpw
b8f36f4fa3c6   wordpress:latest   "docker-entrypoint.s..." 23 minutes ago      Up 23 minutes       80/tcp
wpstack_wordpress.1.n9you5nf9g8pizt54dn1wgh04
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$
```

Task 8: Remove the Stack

```
docker stack rm wpstack
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ |
```

```
docker service ls
```

```
docker ps
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ docker ps
docker service ls
CONTAINER ID   IMAGE                                COMMAND                                CREATED        STATUS        PORTS                                NAMES
AMES          b07bf5fc3d11   sonarqube:lts-community              "/opt/sonarqube/dock..." 13 days ago   Up 34 minutes   0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp   s
onarquibeb6cb118b716   postgres:13                "docker-entrypoint.s..." 13 days ago   Up 34 minutes   5432/tcp                                s
onar-db24fd004ba3a   containerassgn-mybackend      "uvicorn main:app --..." 4 weeks ago   Up 34 minutes   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   c
ontainerassgn_mybackend48bbb6b983c1   containerassgn-mydatabase     "docker-entrypoint.s..." 4 weeks ago   Up 34 minutes (healthy)                                c
ontainerassgn_mydatabase732c618607a6   projectassignment-backend      "docker-entrypoint.s..." 4 weeks ago   Up 34 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   p
roject_node_backendf98234b377c0   projectassignment-database     "docker-entrypoint.s..." 4 weeks ago   Up 34 minutes (healthy)   5432/tcp                                p
project_postgres_db
ID            NAME            MODE            REPLICAS            IMAGE            PORTS
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS/Experiment11$ |
```

PART C – ANALYSIS (Compose vs Swarm)

Docker Compose works on a single host with manual recovery and limited scaling. Docker Swarm supports multi-node clusters, automatic scaling, load balancing, and self-healing. Compose is best for development/testing, while Swarm is suitable for small production.

PART D – IMPORTANT OBSERVATIONS

- Same YAML file works for both Compose and Swarm
- docker compose up -d → Compose mode
- docker stack deploy → Swarm mode
- Swarm manages services, not containers
- Built-in load balancer avoids port conflicts

PART E – LEARNING OUTCOME CHECK

- Why is Compose not enough for production?
- Difference between docker compose up and docker stack deploy
- How does Swarm achieve self-healing?
- What happens when a container is killed?
- Can the same Compose file be reused?

SUMMARY

You progressed from single containers to multi-container applications and now to orchestration using Docker Swarm. You learned scaling, self-healing, and load balancing. Compose defines the application, Swarm runs it reliably.

QUICK REFERENCE COMMANDS

```
docker swarm init
docker stack deploy -c docker-compose.yml <stack-name>
docker service ls
docker service scale <stack-name_service-name>=<replicas>
docker service ps <service-name>
docker stack rm <stack-name>
docker swarm leave --force
```


Containerization_and_DevOps

Experiment 12: Study and Analyse Container Orchestration using Kubernetes

Objective

Learn why Kubernetes is used, understand its core concepts, and perform deployment, scaling, and self-healing using Kubernetes commands.

Why Kubernetes over Docker Swarm?

- Industry Standard → Widely used in companies
- Powerful Scheduling → Automatically assigns workloads
- Large Ecosystem → Supports many tools and plugins
- Cloud-Native → Works with AWS, GCP, Azure

Core Kubernetes Concepts

- Container → Pod (smallest unit in Kubernetes)
- Compose Service → Deployment (defines how app runs)
- Load Balancing → Service (exposes app)
- Scaling → ReplicaSet (maintains pod count)

Hands-On Lab (k3d / Minikube)

Assumption: kubectl and cluster already installed

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

◆ Task 1: Create Deployment

Create `wordpress-deployment.yaml` `apiVersion: apps/v1`

`kind: Deployment`

`metadata:`

`name: wordpress`

`spec:`

`replicas: 2`

`selector:`

`matchLabels:`

`app: wordpress`

`template:`

`metadata:`

`labels:`

`app: wordpress`

`spec:`

`containers:`

`- name: wordpress`

`image: wordpress:latest`

`ports:`

`- containerPort: 80`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ nano wordpress-deployment.yaml
```

▶ Apply: `kubectl apply -f wordpress-deployment.yaml`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
```

◆ Task 2: Expose as Service

Create `wordpress-service.yaml` `apiVersion: v1`

`kind: Service`

`metadata:`

`name: wordpress-service`

`spec:`

`type: NodePort`

`selector:`

`app: wordpress`

`ports:`

`- port: 80`

`targetPort: 80`

`nodePort: 30007`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ nano wordpress-service.yaml
```

▶ Apply: `kubectl apply -f wordpress-service.yaml`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl apply -f wordpress-service.yaml
service/wordpress-service created
```

◆ Task 3: Verify

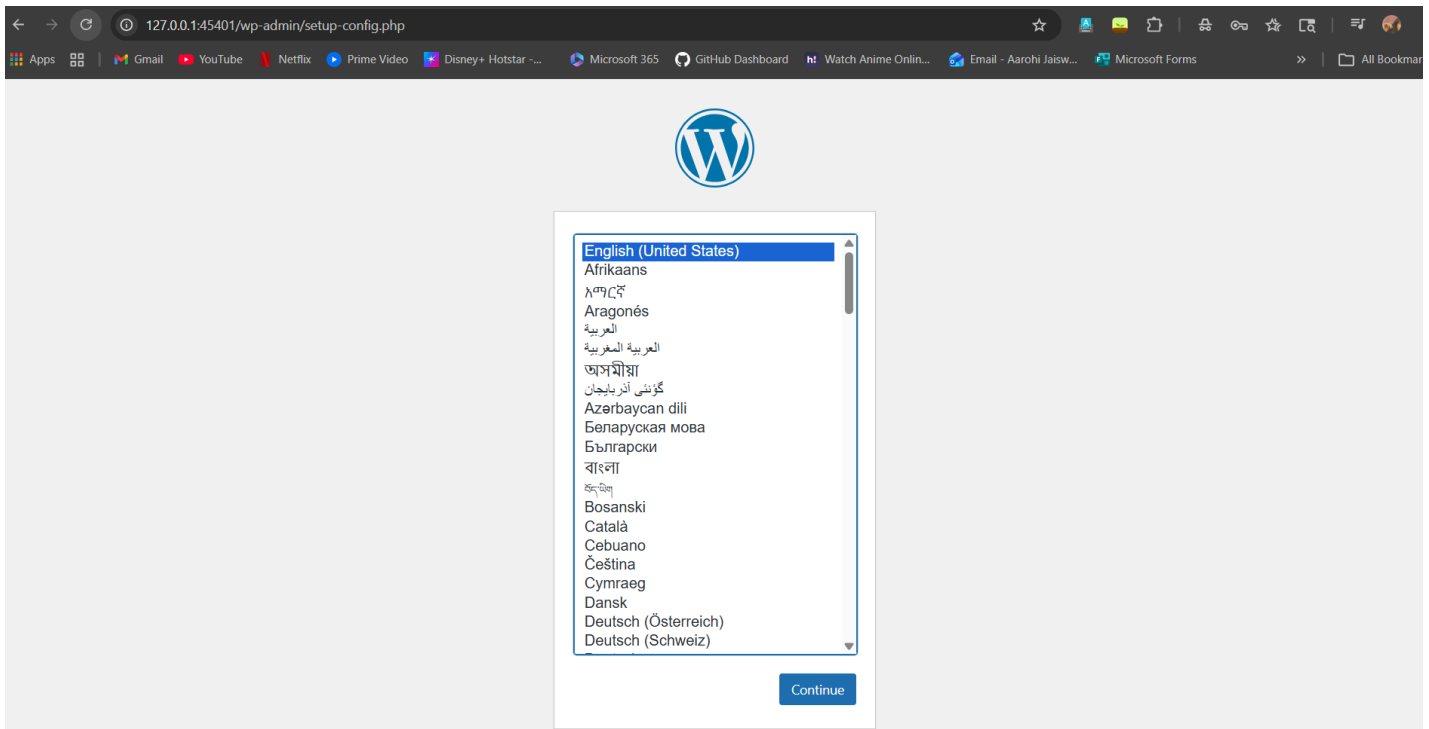
▶ Check pods: `kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-4t6f2         1/1     Running   0           90s
wordpress-6698dd7d66-jmlkx         1/1     Running   0           90s
```

▶ Check service: `kubectl get svc`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1    <none>        443/TCP          10m
wordpress-service   NodePort    10.106.157.6 <none>        80:30007/TCP     6s
```

▶ Access: `http://<node-ip>:30007`



▶ Get IP: `minikube ip`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ minikube ip
192.168.49.2
```

◆ Task 4: Scale Deployment

▶ Scale: `kubectl scale deployment wordpress --replicas=4`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
```

▶ Verify: `kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-457jz	1/1	Running	0	17s
wordpress-6698dd7d66-4t6f2	1/1	Running	0	11m
wordpress-6698dd7d66-jmlkx	1/1	Running	0	11m
wordpress-6698dd7d66-mc86g	1/1	Running	0	17s

◆ Task 5: Self-Healing

▶ Commands: `kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-457jz	1/1	Running	0	27s
wordpress-6698dd7d66-4t6f2	1/1	Running	0	11m
wordpress-6698dd7d66-jmlkx	1/1	Running	0	11m
wordpress-6698dd7d66-mc86g	1/1	Running	0	27s

`kubectl delete pod <pod-name>`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl delete pod wordpress-6698dd7d66-457jz
pod "wordpress-6698dd7d66-457jz" deleted from default namespace
```

`kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-4t6f2	1/1	Running	0	12m
wordpress-6698dd7d66-jdtxg	1/1	Running	0	8s
wordpress-6698dd7d66-jmlkx	1/1	Running	0	12m
wordpress-6698dd7d66-mc86g	1/1	Running	0	96s

Swarm vs Kubernetes

- Swarm → Easy but limited
- Kubernetes → Advanced and industry standard

Tool Usage

- k3d → Quick learning

- Minikube → Local testing
- kubeadm → Production setup

Cheat Sheet

```
kubectl apply -f file.yaml
kubectl get pods
kubectl get svc
kubectl scale deployment <name> --replicas=N
kubectl delete pod <pod-name>
kubectl get nodes
```

Conclusion

- Learned Kubernetes basics
- Deployed WordPress
- Performed scaling
- Tested self-healing
- Understood real cluster setup

 Next: Deploy your own app